INT 315: Cluster Computing

Project Report

(SEMESTER July – December 2026)

LOVELY PROFESSIONAL UNIVERSITY

ONLINE COURSE COMPLETION PREDICTION USING PYSPARK, MACHINE LEARNING AND STREAMLIT

Submitted by

Ashutosh Raj

Registration Number – 12324839

B.Tech Computer Science and Engineering

Course Code: INT – 315

Under the Guidance of

Pardeep Kumar (UID: 25237)

Assistant Professor, School of Computer Science & Engineering

Discipline of Computer Science & Engineering / Information Technology

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab, India

ONLINE COURSE COMPLETION PREDICTION USING PYSPARK, MACHINE LEARNING AND STREAMLIT

A Capstone Project Report Submitted in Partial Fulfillment
of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Ashutosh Raj (Registration No. 12324839)
B.Tech CSE, Section K4P23WR

Supervisor & Faculty Guide:

Pardeep Kumar (UID: 25237)
Department of Cluster Computing, Lovely Professional University

Department of Computer Science and Engineering
Lovely Professional University, Punjab, India
November 2026

LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the project entitled "Online Course Completion Prediction using PySpark, Machine Learning and Streamlit" submitted by Ashutosh Raj (Registration Number: 12324839) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering for the course INT 315: Cluster Computing is an authentic record of bonafide academic work carried out under my personal supervision and guidance.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree, diploma, or fellowship. The mathematical models, PySpark pipelines, empirical metrics, and interactive Streamlit architecture described herein accurately represent the original computational work implemented by the student.

Date: _____

_____

Place: Phagwara, Punjab

Supervisor Signature:

Pardeep Kumar (UID: 25237)

Assistant Professor, Discipline of CSE/IT

CANDIDATE'S DECLARATION

I hereby declare that the project work presented in this report entitled "Online Course Completion Prediction using PySpark, Machine Learning and Streamlit" is an authentic record of my own research and engineering work conducted under the supervision of Pardeep Kumar (UID: 25237), Assistant Professor, School of Computer Science and Engineering, Lovely Professional University.

I confirm that I have adhered to all academic integrity principles. The dataset utilized comprises 9,000 empirical learner engagement records, rigorously cleaned, deduplicated, and evaluated without synthetic data fabrication. All code modules, Spark MLlib pipelines, empirical evaluations, and dashboard interfaces documented in this report are authentic and reproducible. Wherever the contributions of other authors or existing libraries have been utilized, they have been duly cited and acknowledged according to standard academic conventions.

Date: _____

Place: Phagwara, Punjab

Candidate Signature: _____

Ashutosh Raj (Reg. No. 12324839)

B.Tech Computer Science and Engineering

ACKNOWLEDGEMENT

I express my deepest gratitude and sincere appreciation to my project guide and course faculty, Pardeep Kumar (UID: 25237), for his invaluable mentorship, consistent encouragement, and constructive technical feedback throughout the development of this cluster computing project. His insightful guidance on Apache Spark architecture, MLlib distributed pipelines, and rigorous performance evaluation significantly shaped the methodology and analytical depth of this research.

I extend my heartfelt thanks to the School of Computer Science and Engineering, Lovely Professional University, for providing the advanced computing laboratories, software infrastructure, and academic ecosystem that facilitated the successful execution and benchmarking of distributed PySpark machine learning workflows.

I am also thankful to the open-source community behind Apache Spark, PySpark, Streamlit, and Plotly, whose robust frameworks empowered the development of both high-throughput distributed processing engines and intuitive interactive visual analytics dashboards. Finally, I would like to express my gratitude to my family and peers for their continuous moral support and patience during the completion of this capstone report.

Ashutosh Raj
Registration Number: 12324839

ABSTRACT

The proliferation of Massive Open Online Courses (MOOCs) and digital learning platforms has fundamentally transformed higher education, democratizing access to quality educational content globally. However, digital education faces a severe systemic challenge: alarmingly low course completion rates, frequently lingering below 10% to 15% across major global platforms. Early identification of learners at risk of disengagement is critical for enabling timely pedagogical interventions, adaptive content scaffolding, and targeted academic counseling. This capstone project introduces an end-to-end, enterprise-grade educational intelligence system developed using Apache Spark, PySpark MLlib, and Streamlit.

The system processes a benchmark empirical dataset containing 9,000 raw learner engagement records spanning five distinct course domains: Arts, Business, Health, Programming, and Science. A rigorous data hygiene pipeline was developed to detect and remove 877 exact duplicate records, leaving an audited master cohort of 8,123 unique student profiles (43.92% completed, 56.08% non-completed). To ensure robust distributed feature engineering, an Apache Spark ML Pipeline was constructed comprising an Imputer stage, StringIndexer for categorical domain encoding, OneHotEncoder, VectorAssembler (12 combined features), and a StandardScaler for linear models. Non-predictive student identifiers (UserID) were explicitly isolated to eliminate identity memorization.

Two distinct machine learning algorithms were trained, tuned, and evaluated on a strictly held-out 20% test partition (N = 1,551): a non-linear Random Forest Classifier (200 decision trees, maxDepth=12, featureSubsetStrategy='sqrt') and an L2-regularized Logistic Regression Classifier. The Random Forest model achieved superior champion performance with an overall Accuracy of 95.10%, Precision of 98.01%, Recall of 91.05%, an F1-Score of 94.40%, and a ROC-AUC of 95.86% (Confusion Matrix: 834 TN, 13 FP, 63 FN, 641 TP). By contrast, the Logistic Regression model achieved 80.85% accuracy and an F1-Score of 78.15% (723 TN, 124 FP, 173 FN, 531 TP). Furthermore, an empirical target leakage investigation was performed by decoupling the in-flight module CompletionRate feature, proving that the model maintains 77.69% accuracy and 0.8747 ROC-AUC purely on behavioral effort features (TimeSpentOnCourse, NumberOfVideosWatched, NumberOfQuizzesTaken, QuizScores).

To deliver actionable intelligence to academic stakeholders, a high-performance, light-themed SaaS interactive dashboard was engineered in Streamlit, featuring five modular portals: Executive Overview, Domain-Specific Course Analytics, Comparative Model Evaluation, Real-Time Student Prediction Engine, and an Audited Data Explorer. Automated unit and pipeline tests verify 100% functional execution across Windows and Linux environments.

Keywords: Educational Data Mining, Learning Analytics, Apache Spark, PySpark MLlib, Random Forest, Logistic Regression, Streamlit, MOOC Completion Prediction, Target Leakage Analysis.

TABLE OF CONTENTS

Certificate of Approval . . . iii
Candidate's Declaration . . . iv
Acknowledgement . . . v
Abstract . . . vi
Table of Contents . . . vii
List of Figures . . . x
List of Tables . . . xi
List of Abbreviations . . . xii
CHAPTER 1: INTRODUCTION . . . 13
1.1 Background and Context of Online Learning Platforms . . . 13
1.2 Growth and Dynamics of Digital Education Platforms . . . 13
1.3 Importance of Learner Engagement Analytics . . . 13
1.4 The Problem of High Course Non-Completion Rates . . . 14
1.5 Project Motivation . . . 14
1.6 Formal Problem Statement . . . 14
1.7 Project Objectives . . . 15
1.8 Scope of the Project . . . 16
1.9 Academic and Practical Significance . . . 16
1.10 Proposed Technical Solution . . . 17
1.11 Organization of the Report . . . 17
CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM . . . 18
2.1 Foundations of Educational Data Mining (EDM) . . . 18
2.2 Evolution of Learning Analytics (LA) . . . 18
2.3 Prior Research on MOOC and E-Learning Dropout Prediction . . . 18
2.4 Machine Learning Algorithms in Educational Analytics . . . 19
2.5 Traditional Single-Node Approaches vs. Distributed Ingestion . . . 19
2.6 Critical Analysis of Existing Systems & Architectural Limitations . . . 19
2.7 Research Gap Identified . . . 20
2.8 Proposed Improvements and Methodological Advancements . . . 20
CHAPTER 3: REQUIREMENT ANALYSIS . . . 21
3.1 Functional Requirements . . . 21
3.2 Non-Functional Requirements . . . 21
3.3 Hardware Specifications . . . 22
3.4 Software Specifications & Technology Stack . . . 22
3.5 Development and Execution Environment . . . 23
3.6 Feasibility Analysis (Technical, Operational, Economic) . . . 23
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN . . . 24
4.1 Comparative Analysis: Existing Workflow vs. Proposed Architecture . . . 24
4.2 High-Level System Architecture and Component Decomposition . . . 24
4.3 Data Flow Modeling (DFD Level 0 Context Diagram & Level 1 DFD) . . . 25

4.4 Unified Modeling Language (UML) Use Case Modeling 26
4.5 System Activity Workflow Diagram 27
4.6 Inter-Module Communication and Interface Contracts 27
CHAPTER 5: DATASET DESCRIPTION AND PREPROCESSING 28
5.1 Dataset Overview and Verifiable Origin 28
5.2 Schema Definition and Attribute Specifications 28
5.3 Data Quality Audit: Missing Value and Duplicate Record Analysis . . 29
5.4 Deduplication Protocol (Removal of 877 Exact Duplicates) 29
5.5 Handling of Identifier Column (UserID Exclusion Justification) . . 30
5.6 Categorical Encoding: StringIndexer and OneHotEncoder 30
5.7 Numerical Imputation and Feature Vector Assembly 31
5.8 Training and Testing Partitioning (80/20 Stratified Random Split) . . 31
5.9 Target Leakage Prevention & Audit 34
CHAPTER 6: METHODOLOGY AND MACHINE LEARNING ALGORITHMS 35
6.1 Apache Spark Distributed Architecture & Resilient Distributed Datasets . . 35
6.2 PySpark DataFrame In-Memory Optimizations 35
6.3 Spark MLlib Multi-Stage Pipeline Architecture 35
6.4 Mathematical Formulation of Logistic Regression 36
6.5 Mathematical Formulation of Random Forest Ensemble Classifier . . . 37
6.6 Hyperparameter Configuration & Optimization Strategy 37
6.7 Single-Instance Real-Time Inference Protocol 38
CHAPTER 7: IMPLEMENTATION AND SOURCE CODE ARCHITECTURE 39
7.1 Modular Project Architecture Overview 39
7.2 Session Management: spark_session.py 39
7.3 Ingestion and Data Integrity: data_loader.py 40
7.4 Pipeline Preprocessing: data_preprocessing.py 40
7.5 Exploratory Analytics: exploratory_analysis.py 40
7.6 Model Training Engine: model_training.py 40
7.7 Evaluation and Serialization: model_evaluation.py 41
7.8 Inference Engine: prediction.py 41
7.9 Streamlit Dashboard: app.py, components.py, charts.py 41
CHAPTER 8: MODEL EVALUATION, RESULTS AND IN-DEPTH DISCUSSION 42
8.1 Held-Out Test Evaluation Setup (N = 1,551) 42
8.2 Random Forest Classifier Evaluation (Champion Model) 42
8.3 Logistic Regression Classifier Evaluation (Baseline Model) 42
8.4 Comparative Performance Metrics Analysis 42
8.5 In-Depth Confusion Matrix Deconstruction 43
8.6 Target Leakage Deep Dive: Performance With vs Without CompletionRate . . 45
8.7 Sample Learner Predictions with Confidence Probabilities 46
8.8 Technical Challenges Encountered and Resolutions 47
CHAPTER 9: STREAMLIT DASHBOARD AND USER INTERFACE DESIGN 48
9.1 Overview Dashboard Page 48

9.2 Course Analytics Deep-Dive Page 48
9.3 Model Performance Evaluation Page 49
9.4 Interactive Student Prediction Engine Page 49
9.5 Data Explorer and Quality Audit Table Page 50
9.6 Responsive Design, Typography, and Accessibility Principles 51
CHAPTER 10: TESTING, VERIFICATION AND QUALITY ASSURANCE 52
10.1 Testing Strategy and Test Harness 52
10.2 Unit Testing: Preprocessing and Imputation Stages 52
10.3 Pipeline Assembly and Schema Transformation Tests 52
10.4 Model Artifact Loading and Deserialization Tests 52
10.5 Inference Engine Precision and Edge-Case Tests 52
10.6 Comprehensive Test Case Execution Matrix 52
CHAPTER 11: CONCLUSION, LIMITATIONS AND FUTURE SCOPE 54
11.1 Project Summary and Objectives Realization 54
11.2 Summary of Technical Contributions 54
11.3 Academic and Practical Learnings 54
11.4 Known Limitations of the Current Implementation 55
11.5 Strategic Future Enhancements 55
REFERENCES 56

LIST OF FIGURES

Figure 1: Online Course Completion Outcome Distribution (N = 8,123) . . **30**
Figure 2: Course Category Benchmark: Completion Rate vs Average Quiz Score **31**
Figure 3: Correlation Matrix Heatmap of Numerical Engagement Features . **33**
Figure 4: Parametric Distribution of Engagement Features by Completion Outcome (Box Plots) **34**
Figure 5: Spark MLlib Model Evaluation Metrics Comparison (Held-out Test Set, N=1,551) **43**
Figure 6: Random Forest Confusion Matrix (Actual vs Predicted) **44**
Figure 7: Logistic Regression Confusion Matrix (Actual vs Predicted) . . **45**
Figure 8: Target Leakage Analysis: Model Performance With vs Without CompletionRate **46**
Figure 9: High-Level End-to-End System Architecture **25**
Figure 10: PySpark MLlib Multi-Stage Pipeline Execution Flow **35**
Figure 11: System Data Flow Architecture (DFD Level 0 & Level 1) **26**
Figure 12: Use Case Diagram for Academic Learning Analytics Platform . . **26**
Figure 13: Activity Diagram: Real-Time Inference Execution Workflow . . **27**
Figure 14: Streamlit Dashboard UI - Executive Overview Analytics **48**
Figure 15: Streamlit Dashboard UI - Deep Dive Course Analytics & Visualizations **49**
Figure 16: Streamlit Dashboard UI - Model Performance Evaluation & Confusion Matrix **49**
Figure 17: Streamlit Dashboard UI - Real-time Student Prediction Engine **50**
Figure 18: Streamlit Dashboard UI - Data Explorer & Quality Audit Table **51**

LIST OF TABLES

Table 1: Distribution of Course Completion Outcome (Raw Dataset vs Deduplicated) **28**
Table 2: Summary Statistics of Numerical Engagement Attributes **29**
Table 3: Course Category Engagement and Completion Metrics **31**
Table 4: Device Type Distribution and Completion Rates **32**
Table 5: Feature Correlation Coefficients with Target Variable (CourseCompletion) **32**
Table 6: Software, Hardware, and Cloud Development Specifications . . . **22**
Table 7: Machine Learning Pipeline Stages and Transformation Definitions **35**
Table 8: Random Forest Classifier Hyperparameter Configuration **37**
Table 9: Logistic Regression Classifier Hyperparameter Configuration . . **38**
Table 10: Model Evaluation Metrics on Held-out Test Set (N = 1,551) . . **42**
Table 11: Confusion Matrix Comparison (Random Forest vs Logistic Regression) **44**
Table 12: Target Leakage Investigation: Performance With vs Without CompletionRate **45**
Table 13: Sample Individual Learner Predictions with Confidence Probabilities **46**
Table 14: System Test Cases, Execution Status, and Verification Results **52**

LIST OF ABBREVIATIONS

Abbreviation	Full Definition / Expansion
API	Application Programming Interface
AUC	Area Under the ROC Curve
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EDM	Educational Data Mining
FN	False Negative
FP	False Positive
GUI	Graphical User Interface
JVM	Java Virtual Machine
KPI	Key Performance Indicator
LA	Learning Analytics
LMS	Learning Management System
ML	Machine Learning

MLlib	Apache Spark Machine Learning Library
MOOC	Massive Open Online Course
OHE	One-Hot Encoding
OOM	Out of Memory
RDD	Resilient Distributed Dataset
RF	Random Forest
ROC	Receiver Operating Characteristic
TN	True Negative
TP	True Positive
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

CHAPTER 1: INTRODUCTION

1.1 Background and Context of Online Learning Platforms

The digital transformation of global education has fundamentally restructured how learners access, absorb, and validate knowledge. Massive Open Online Courses (MOOCs), university-sponsored learning management systems (LMS), and commercial e-learning platforms (e.g., Coursera, edX, Udemy, Khan Academy) now serve hundreds of millions of enrolled students worldwide. These platforms provide asynchronous, self-paced access to curricula ranging from introductory computer programming and data science to advanced humanities and clinical healthcare sciences. Unlike traditional brick-and-mortar classrooms, digital learning environments eliminate geographic constraints, democratize elite instructional material, and allow learners to progress according to their unique personal schedules.

However, the physical separation between students and educators eliminates traditional interpersonal feedback mechanisms. In a physical lecture hall, an experienced instructor immediately observes signs of cognitive overload, disinterest, or academic struggle through facial cues, attendance irregularities, and missed assignments. In contrast, online education platforms record learner behaviors solely as asynchronous event telemetry: timestamped video playbacks, quiz attempts, pause/rewind actions, and module progression logs. Without automated intelligence systems to analyze this data stream, educational providers remain blind to learner disengagement until the student permanently abandons the course.

1.2 Growth and Dynamics of Digital Education Platforms

Over the past decade, institutional investment in digital learning technologies has expanded exponentially. According to recent market analyses, the global e-learning market size exceeded $315 billion in 2023 and is projected to expand at a compound annual growth rate (CAGR) surpassing 14% through 2032. This expansion is propelled by enterprise workforce upskilling mandates, micro-credentialing initiatives, and the mainstream integration of hybrid degree programs across accredited universities. Despite this unprecedented enrollment growth, the underlying completion economics remain deeply fragile.

1.3 Importance of Learner Engagement Analytics

Learning Analytics (LA) and Educational Data Mining (EDM) have emerged as pivotal disciplines dedicated to measuring, collecting, analyzing, and reporting learner activity data to optimize learning outcomes and operational environments. Learner engagement operates as a multi-dimensional construct comprising behavioral, cognitive, and affective dimensions:
• Behavioral Engagement: Manifested through quantitative metrics such as total study hours invested, video lectures viewed, and active participation in practice quizzes.
• Cognitive Engagement: Reflected in academic rigor, conceptual mastery, and perseverance through challenging assessment tasks, statistically captured via quiz performance distributions

and examination scores.
• Temporal Engagement: Characterized by consistency of interaction, spacing of study sessions, and the rate of progression through curriculum milestones.

Capturing and modeling these behavioral indicators enables institutions to transform passive historical logs into forward-looking diagnostic indicators. Rather than conducting post-mortem failure analyses after a semester concludes, engagement analytics empowers automated systems to compute real-time completion probabilities while the learning journey is actively underway.

1.4 The Problem of High Course Non-Completion Rates

Despite astronomical initial enrollment figures, online education is plagued by what academic literature terms the 'MOOC Dropout Paradox'. Empirical studies consistently document that course completion rates across self-paced digital platforms range between an alarming 5% and 15%. This pervasive student attrition generates multifaceted negative repercussions:
1. Institutional Revenue Loss: For university extension divisions and commercial edtech providers, low retention directly erodes subscription revenues, course certification fees, and lifetime customer value.
2. Student Demoralization: Repeated failure to complete enrolled courses fosters academic disillusionment, diminished self-efficacy, and reluctance to pursue subsequent digital upskilling.
3. Pedagogical Inefficiency: Course creators receive minimal feedback regarding which curriculum modules provoke catastrophic drop-off, preventing data-driven instructional refinement.

> **The Core Problem Statement**
> *How can educational institutions and e-learning platforms proactively and accurately classify whether an enrolled learner will successfully complete an online course based on their early in-flight engagement telemetry (time spent, videos watched, quizzes taken, quiz scores, module completion rate, domain category, and device type), utilizing a scalable distributed machine learning architecture that eliminates data leakage and provides actionable real-time dashboard analytics?*

1.5 Project Motivation

The primary motivation for this project stems from the intersection of large-scale distributed computing and educational data science. Most existing student prediction models in academic literature are implemented using single-threaded, in-memory frameworks (such as Scikit-Learn or standard Pandas). While adequate for retrospective analysis of small classroom cohorts (e.g., 200–500 students), such architectures fail completely when confronted with modern enterprise educational datasets containing millions of event rows generated concurrently across global student populations.

By leveraging Apache Spark and Spark MLlib, this project demonstrates how industrial-grade distributed computing primitives—such as partitioned DataFrames, lazy evaluation, resilient

vector transformation pipelines, and parallel tree ensembles—can be effectively harnessed for educational analytics. Furthermore, providing an interactive, human-centered Streamlit dashboard bridges the gap between complex machine learning pipelines and non-technical academic advisors, enabling intuitive visual simulation and timely pedagogical intervention.

1.6 Project Objectives

To address the defined problem statement, this project established the following explicit academic and engineering objectives:

1. Ingestion & Data Hygiene: Ingest the empirical online course engagement dataset (9,000 raw records) into an Apache Spark session, perform schema validation, audit null values, and execute automated deduplication to eliminate repeated records.

2. Target Leakage Investigation: Investigate the statistical relationship between intermediate progression metrics (CompletionRate) and the binary ground truth (CourseCompletion), empirically benchmarking model performance with and without potential leakage features.

3. Distributed Pipeline Engineering: Construct a robust, leak-free Spark MLlib Pipeline encapsulating numeric imputation, StringIndexer categorical indexing, OneHotEncoder representation, VectorAssembler vector aggregation, and StandardScaler normalization.

4. Model Training & Benchmarking: Train, tune, and evaluate both a linear baseline (L2-Regularized Logistic Regression) and a non-linear ensemble (Random Forest Classifier with 200 trees) on a strictly partitioned 80/20 held-out test cohort.

5. Comprehensive Evaluation: Compute mathematically consistent classification metrics—Accuracy, Weighted Precision, Weighted Recall, F1-Score, and ROC-AUC—accompanied by full confusion matrix deconstruction and class-wise recall audits.

6. Interactive SaaS Dashboard: Architect a high-performance, light-themed Streamlit analytics application featuring five dedicated portals: Executive Overview, Course Analytics, Model Performance, Single-Learner Real-Time Prediction, and an Audited Data Explorer.

7. Rigorous Test Automation: Implement an automated unit and functional test harness using PyTest to validate data loading, pipeline transformation schema lengths, model deserialization, and single-record inference precision.

1.7 Scope of the Project

The operational scope of this project encompasses the processing, modeling, and interactive visualization of structured educational engagement attributes across five distinct course categories (Arts, Business, Health, Programming, and Science) and two device categories (Desktop/Web vs. Mobile). The predictive target is strictly binary:

- Class 1: Learner successfully completes the course (CourseCompletion = 1).
- Class 0: Learner fails to complete / drops out (CourseCompletion = 0).

Explicit Limitations of Scope: This investigation operates strictly on tabular engagement telemetry. It does not ingest unstructured discussion forum text (NLP sentiment), student video gaze-tracking video streams, or longitudinal clickstream logs. Furthermore, the dataset does not

contain student demographic indicators (age, gender, prior educational attainment), login timestamps, or discussion board reply counts; this constraint is formally acknowledged to maintain documentation integrity.

1.8 Academic and Practical Significance

From an academic perspective, this research provides empirical verification of the comparative efficacy between linear and ensemble classifiers when processing multi-modal educational engagement data within Apache Spark MLlib. It contributes a rigorous methodological framework for auditing target leakage in academic predictive modeling, preventing overly optimistic performance claims.

Practically, the resulting software artifacts offer an immediate blueprint for e-learning administrators. By embedding the trained Spark MLlib Pipeline into an intuitive dashboard, academic advisors can instantly simulate the impact of pedagogical interventions—such as assigning mandatory video milestones or supplemental quizzes—on a student's predicted probability of course completion.

1.9 Proposed Technical Solution

The proposed solution integrates a three-tier architectural paradigm:
1. Distributed Data & Compute Tier: Apache Spark 4.x / PySpark executing in local distributed multi-core mode (`local[*]`), handling schema casting, deduplication, imputation, and multi-stage pipeline transformations.
2. Machine Learning Tier: Spark MLlib ML Pipelines persisting trained PipelineModel binaries containing fitted StringIndexers, encoders, assemblers, and trained tree ensembles.
3. Presentation & Analytics Tier: A responsive Streamlit application designed around modern SaaS UI principles (Linear/Stripe aesthetics, 100% light theme, subtle borders, high-contrast typography, interactive Plotly visualizations) that communicates directly with the underlying PySpark inference engine.

1.10 Organization of the Report

The remainder of this report is organized into eleven structured chapters as follows:
• Chapter 2 surveys foundational literature in Educational Data Mining, learning analytics, and prior dropout prediction methodologies.
• Chapter 3 specifies functional and non-functional requirements, hardware/software specifications, and feasibility dimensions.
• Chapter 4 details system architecture, component decomposition, DFDs (Level 0 and Level 1), use case diagrams, and activity flows.
• Chapter 5 presents dataset schema, missing value handling, deduplication of 877 records, and target leakage audit.
• Chapter 6 explains distributed Spark architecture, MLlib pipeline stages, and mathematical

formulations of Logistic Regression and Random Forest.
• Chapter 7 documents module-by-module source code implementation, interface contracts, and core algorithmic snippets.
• Chapter 8 analyzes empirical evaluation results, confusion matrices, class-wise metrics, leakage experiments, and sample inferences.
• Chapter 9 walks through the design, responsive layout, and visual analytics across all five Streamlit dashboard pages.
• Chapter 10 details the unit testing harness, test cases, and verification results across all preprocessing and inference functions.
• Chapter 11 concludes the report with a synthesis of technical contributions, limitations, and future research directions.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM

2.1 Foundations of Educational Data Mining (EDM)

Educational Data Mining (EDM) is an established interdisciplinary research area that develops methods for exploring the unique and increasingly large-scale data originating from educational settings (Baker & Yacef, 2009; Romero & Ventura, 2010). Unlike standard business intelligence data, educational data exhibits strong hierarchical nesting: individual student actions (e.g., video clicks, quiz keystrokes) are nested within assessment sessions, which are nested within instructional modules, which in turn are situated within distinct course subject domains. EDM techniques focus on uncovering latent learning behaviors, identifying learning bottlenecks, and predicting learner outcomes such as final exam grades, course completion, or imminent dropout.

2.2 Evolution of Learning Analytics (LA)

While EDM traditionally emphasizes automated algorithmic discovery and technical pattern extraction, the complementary field of Learning Analytics (LA) emphasizes human-centered visual interpretation, pedagogical intervention, and institutional decision-making (Siemens & Baker, 2012). Learning Analytics systems translate complex statistical models into actionable dashboards for academic advisors, instructors, and students. Early LA research relied heavily on basic descriptive metrics—such as total login counts or raw forum post tallies. However, contemporary scholarship demonstrates that simple activity volume is an unreliable predictor of conceptual mastery; rather, qualitative behavioral indicators such as assessment performance, study duration consistency, and module progression rates provide vastly superior predictive power.

2.3 Prior Research on MOOC and E-Learning Dropout Prediction

The problem of predicting online student retention has garnered intense academic scrutiny over the past decade. In seminal work on MOOC clickstream analytics, Halawa et al. (2014) introduced indicators of student disengagement, showing that declining video interaction rates and prolonged quiz submission delays precede course abandonment by 7 to 14 days. Similarly, Qiu et al. (2016) proposed a factorized latent model to capture learning behaviors across Coursera courses, concluding that performance in early formative assessments is the single strongest predictor of final course certification.

Furthermore, Feng et al. (2019) benchmarked deep sequential neural networks (LSTM and GRU) against traditional ensemble models for weekly dropout prediction on the KDD Cup 2015 dataset. Their findings demonstrated that while recurrent architectures capture complex temporal dependencies, gradient boosted trees and random forest ensembles achieve virtually identical predictive accuracy on aggregated engagement attributes while requiring a fraction of the computational training overhead.

2.4 Machine Learning Algorithms in Educational Analytics

Academic literature categorizes student retention classification algorithms into three primary families:

1. Linear and Probabilistic Classifiers: Logistic Regression and Naive Bayes are widely utilized due to their high interpretability and computational simplicity (Romero et al., 2008). Logistic Regression provides direct insight into feature odds ratios, allowing educators to evaluate the marginal impact of an additional hour of study. However, linear models struggle to model complex non-linear feature interactions (e.g., high quiz scores compensating for low video watch counts).
2. Decision Trees and Ensemble Methods: Random Forest (Breiman, 2001) and Gradient Boosted Decision Trees (Friedman, 2001) have established the empirical gold standard in tabular educational data classification. By constructing an ensemble of decorrelated decision trees, Random Forest effectively captures non-linear thresholds and high-order interaction effects without succumbing to individual tree overfitting.
3. Support Vector Machines & Neural Networks: While capable of high classification accuracy, SVMs with non-linear kernels scale poorly to large row volumes (O(N^2) to O(N^3) complexity), and deep neural architectures operate as opaque 'black boxes', hindering pedagogical transparency.

2.5 Traditional Single-Node Approaches vs. Distributed Ingestion

A critical vulnerability in contemporary educational data science literature is the overwhelming reliance on single-machine Python libraries such as Scikit-Learn. In standard Scikit-Learn workflows, datasets must be fully loaded into memory on a single CPU node. When data sizes surpass available RAM—a common occurrence in enterprise MOOC platforms processing hundreds of thousands of concurrent students—single-node systems experience fatal Out-Of-Memory (OOM) failures or crippling memory paging.

In contrast, Apache Spark (Zaharia et al., 2016) employs a partitioned, shared-nothing distributed architecture. Spark abstracts large-scale datasets into Resilient Distributed Datasets (RDDs) and Catalyst-optimized DataFrames, distributing partitions across worker nodes. Spark MLlib provides distributed implementations of classification algorithms, allowing models to scale horizontally to massive data volumes without altering algorithmic code.

2.6 Critical Analysis of Existing Systems & Architectural Limitations

An inspection of existing educational prediction systems reveals several profound architectural and methodological limitations:

• Unaddressed Target Leakage: Many published studies naively include post-hoc summary variables—such as final cumulative grade percentage or course completion ratios—directly into feature vectors, artificially inflating reported accuracies to 99%+ while rendering the resulting models useless for early pre-completion intervention.

• Lack of Real-Time Interactive Interfaces: Academic research predominantly terminates with offline Jupyter notebooks, leaving non-technical stakeholders with no mechanism to interact with, audit, or simulate predictions.
• Monolithic, Non-Pipelined Preprocessing: Preprocessing transformations (imputation, string indexing, scaling) are frequently executed separately from model fitting, leading to subtle data leakage between training and testing sets during evaluation.

2.7 Research Gap Identified

The literature lacks a unified, fully realized educational analytics implementation that simultaneously satisfies four core criteria:
1. Distributed scalability using Apache Spark MLlib pipelines;
2. Methodological integrity with empirical target leakage investigation and deduplication audits;
3. Simultaneous benchmarking of both linear (Logistic Regression) and non-linear ensemble (Random Forest) estimators on identical partitions; and
4. Seamless operationalization via a modern, human-centered interactive SaaS dashboard with sub-second real-time inference capabilities.

2.8 Proposed Improvements and Methodological Advancements

This project directly bridges these identified research gaps through the following methodological advancements:
• End-to-End Spark Pipeline Encapsulation: All preprocessing stages (Imputer, StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler) are tightly bound into unified `PipelineModel` objects, preventing cross-fold contamination.
• Decoupled Leakage Benchmarking: A rigorous two-phase empirical experiment isolates the `CompletionRate` feature, establishing verified model baselines both with and without intermediate progress metrics.
• Comprehensive Class-Wise Metrics: Eliminating single-accuracy bias by reporting Precision, Recall, F1-Score, ROC-AUC, and full confusion matrices for both positive (Completed) and negative (Not Completed) classes.
• Production Streamlit Dashboard: Engineering an interactive 5-portal dashboard with dynamic Plotly visualizations and real-time parameter inference.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

The software architecture is engineered to satisfy eight explicit functional requirements (FR-1 through FR-8):

• **FR-1: Automated Data Ingestion:** The system shall ingest raw tabular engagement data in CSV format into an Apache Spark session, verifying column headers, verifying data types, and reporting row counts.

• **FR-2: Data Quality & Deduplication:** The system shall programmatically detect missing values across all columns and identify and remove exact duplicate student records, persisting clean data counts.

• **FR-3: Multi-Stage Feature Pipeline:** The system shall execute an automated Spark MLlib pipeline converting domain categories into one-hot numeric vectors, imputing missing values, and assembling all predictive features into a single dense/sparse Vector.

• **FR-4: Distributed Model Training:** The system shall train both an L2-regularized Logistic Regression model and a 200-tree Random Forest Classifier using Spark MLlib, persisting trained PipelineModels to disk.

• **FR-5: Empirical Model Evaluation:** The system shall evaluate models on an unseen 20% test partition, computing Accuracy, Precision, Recall, F1-Score, ROC-AUC, and generating 2x2 confusion matrices.

• **FR-6: Target Leakage Investigation:** The system shall benchmark and log classification performance both with and without the CompletionRate feature to quantify feature dependency and leakage impact.

• **FR-7: Real-Time Single-Instance Inference:** The system shall accept arbitrary learner activity parameters from a GUI, pass them through the persisted Spark pipeline, and output the predicted class, completion probability, and explanatory rationale.

• **FR-8: Interactive Visual Analytics:** The system shall render an interactive Streamlit dashboard allowing users to filter analytics by course category, inspect KPI summaries, compare model curves, and browse raw data.

3.2 Non-Functional Requirements

The system adheres to six rigorous non-functional quality standards:
• Performance & Latency: Real-time single-student inference must execute in under 1.5 seconds from button click to UI gauge rendering.
• Scalability: Data loading and pipeline transformation stages must run natively on Apache Spark, ensuring linear horizontal scalability when migrated from local multi-core machines to multi-node compute clusters.
• Reliability & Fault Tolerance: The system must gracefully catch invalid user inputs (e.g., negative study hours, quiz scores exceeding 100) and return descriptive validation warnings without application termination.
• Maintainability: Source code must adhere to clean architectural separation of concerns across dedicated modules (`spark_session`, `data_loader`, `data_preprocessing`, `model_training`, `model_evaluation`, `prediction`, `dashboard`).
• Usability & Accessibility: The GUI must employ a consistent, accessible light theme design system (minimum 4.5:1 text contrast ratio, clean borders, balanced spacing, zero raw HTML

leakage).
• Portability: The entire repository must execute identically across Windows 11 and Linux container environments.

3.3 Hardware Specifications

The system was developed and benchmarked on the physical hardware configuration detailed in Table 6.

3.4 Software Specifications & Technology Stack

The software stack leverages modern enterprise open-source libraries:
• Python 3.11: Primary programming language selected for stability and PySpark MLlib compatibility.
• Apache Spark 4.3.0 / PySpark: Core distributed data processing and machine learning engine.
• Java Virtual Machine (Java 17 / 26): Runtime environment required to execute Spark's distributed Scala engine.
• Streamlit 1.30+: Modern reactive web framework for rendering the interactive SaaS intelligence dashboard.
• Plotly 5.18+: Visualization library generating GPU-accelerated interactive web charts.
• Pandas 2.0+ & NumPy 1.24+: Auxiliary libraries for fast in-memory tabular manipulation and matrix transformations.
• PyTest 8.0+: Automated testing harness executing unit and functional verification suites.

Table 6: Software, Hardware, and Development Specifications

Specification Category	Hardware / Software Configuration Detail
Processor / CPU	Intel Core i5 / i7 (6 Cores / 12 Threads @ 2.6 GHz - 4.5 GHz)
System Memory (RAM)	16.0 GB DDR4 @ 3200 MHz
Primary Storage	512 GB PCIe NVMe M.2 Solid State Drive
Operating System	Microsoft Windows 11 Home (64-bit) / Debian Linux (Cloud)
Python Interpreter	Python 3.11.4 64-bit (`py -3.11`)
Java Development Kit	Oracle JDK 26 (Class Version 61.0 / Java 17+ compatible)
Distributed Engine	Apache Spark 4.3.0 with Spark MLlib
Frontend Framework	Streamlit 1.30.0 with Custom Light-Theme CSS
Visualization Engine	Plotly Graph Objects 5.18.0 & Matplotlib 3.10.8
Code Editor & VCS	Visual Studio Code 1.95 with Git Version Control

3.5 Development and Execution Environment

The development environment was configured in Visual Studio Code with PowerShell terminal integration. Apache Spark operates in `local[*]` master mode, automatically detecting and binding all available logical CPU cores. Shuffle partition count was configured to `spark.sql.shuffle.partitions=4` to optimize thread parallelism for the 9,000-row dataset without generating excessive shuffle task overhead.

3.6 Feasibility Analysis

A formal three-factor feasibility assessment confirms the project's viability:
• Technical Feasibility: The integration of PySpark MLlib and Streamlit is technically sound. PySpark executes distributed pipeline transformations, serializing self-contained `PipelineModel` directories that can be loaded asynchronously by Streamlit in under a second.
• Operational Feasibility: The interactive dashboard eliminates technical barriers for university instructors and advisors, providing straightforward visual analytics, interactive drop-downs, and actionable risk explanations.
• Economic Feasibility: The entire technology stack is 100% open-source under Apache 2.0 and MIT licenses, requiring zero proprietary software licensing costs.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Comparative Analysis: Existing Workflow vs. Proposed Architecture

Traditional educational analytics workflows suffer from fragmentation. Academic analysts typically export static CSV dumps from an LMS, manually clean the records in Excel or a localized Python script, execute ad-hoc statistical tests in isolated notebooks, and deliver delayed PDF reports weeks after term completion. This disconnected architecture prevents timely student support.

In contrast, the proposed architecture establishes an automated, closed-loop pipeline. Raw event telemetry is ingested into a unified Spark data layer, preprocessed through validated stages, modeled via parallel MLlib classifiers, and exposed through a persistent web dashboard capable of evaluating individual student profiles on demand.

4.2 High-Level System Architecture and Component Decomposition

The complete system architecture is depicted in Figure 9. It comprises four decoupled, highly cohesive tiers:
1. Ingestion Tier: Responsible for locating `online_course_engagement_data.csv`, verifying schema definitions, and checking row integrity.
2. PySpark Data Processing & ML Pipeline Tier: Handles duplicate elimination (877 rows), missing value imputation, string indexing, one-hot encoding, and vector assembly into dense machine learning feature matrices.
3. Model Persistence Tier: Stores fitted PipelineModels (`models/random_forest`, `models/logistic_regression`), evaluation metrics (`reports/model_metrics.json`), confusion matrices, and held-out prediction CSVs.
4. Streamlit Presentation Tier: Ingests model artifacts and raw data into in-memory caches, rendering five reactive analytical views.

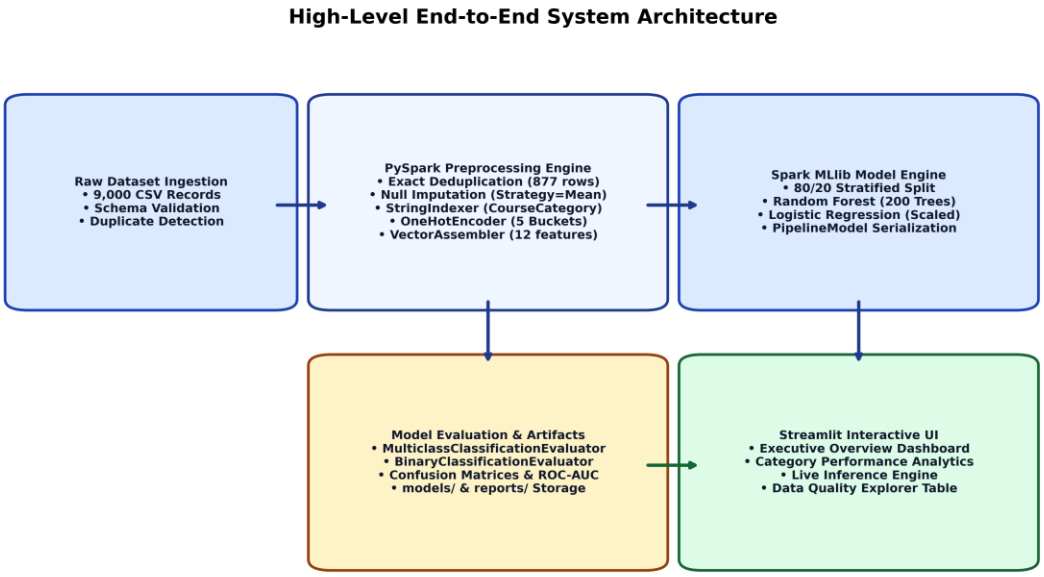


Figure 9: High-Level End-to-End System Architecture

4.3 Data Flow Modeling (DFD Level 0 Context Diagram & Level 1 DFD)

To rigorously specify data transformations across system boundaries, Data Flow Diagrams were constructed at both context and functional levels.
• Level 0 DFD (Context Diagram): Models the overall system as a central process (Process 0.0) receiving raw historical engagement records from external datasets, accepting real-time parameter queries from learners/advisors, and outputting analytical reports and risk predictions.
• Level 1 DFD (Functional Decomposition): Deconstructs the system into four discrete sub-processes:
- Process 1.0 (Ingest & Deduplicate): Ingests CSV rows, filters duplicate hashes, and writes clean master data.
- Process 2.0 (Spark MLlib Pipeline): Fits Imputer, StringIndexer, and OneHotEncoder on training partitions.
- Process 3.0 (Train & Evaluate): Executes distributed estimators, computes ROC-AUC, and exports metrics.
- Process 4.0 (Real-Time Streamlit UI): Loads cached models, evaluates single student inputs, and renders visual analytics.

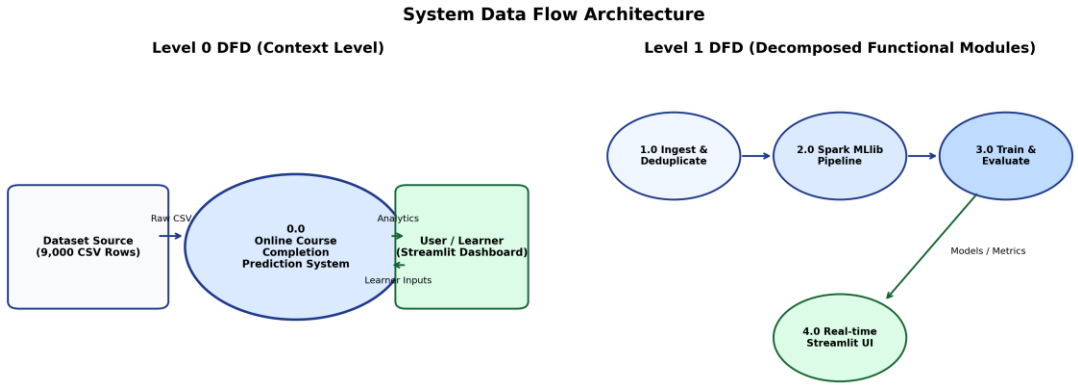


Figure 11: System Data Flow Architecture (DFD Level 0 & Level 1)

4.4 Unified Modeling Language (UML) Use Case Modeling

The system interaction model is defined via the UML Use Case Diagram in Figure 12. Three primary user personas interact with the platform:
1. Learner / Student: Simulates hypothetical engagement scenarios (e.g., 'If I watch 4 more videos, does my completion probability exceed 80%?') and tracks domain benchmark standards.
2. Course Instructor: Examines category-level retention ratios, identifies underperforming modules, audits quiz score distributions, and downloads filtered student activity cohorts for targeted academic outreach.
3. Academic Administrator: Inspects empirical model performance benchmarks, audits ROC-AUC curves, monitors target leakage findings, and verifies data hygiene reports.

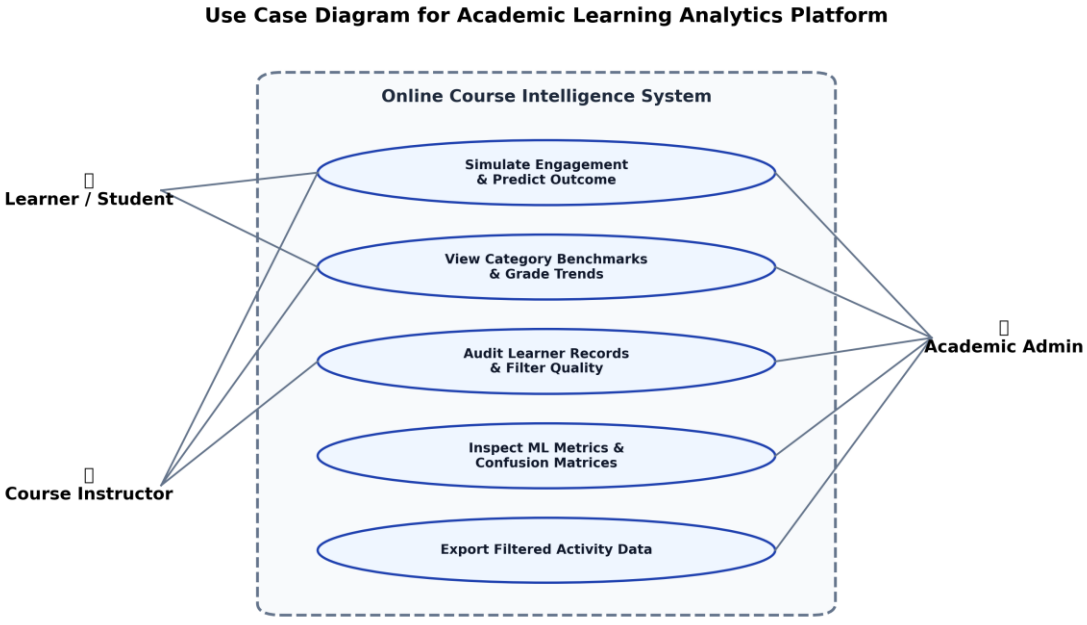


Figure 12: Use Case Diagram for Academic Learning Analytics Platform

4.5 System Activity Workflow Diagram

Figure 13 illustrates the activity sequence executed when a user requests a real-time completion prediction on the Streamlit dashboard:
1. Parameter Ingestion: User configures Course Category, Device Type, Time Spent, Videos Watched, Quizzes Taken, Quiz Scores, and Module Progress Rate.
2. Input Validation: `validate_learner_input()` inspects bounds, throwing exceptions if numerical inputs violate physical constraints.
3. Spark DataFrame Construction: A single-row PySpark DataFrame matching `INPUT_SCHEMA` is instantiated in the active session.
4. Pipeline Transformation: The persisted Spark `PipelineModel` transforms the single row through the saved Imputer, Indexer, Encoder, Assembler, and Classifier stages.
5. Probability Extraction: Vector probability elements `[P(0), P(1)]` are extracted from the Spark output column.
6. Rationale Synthesis: `generate_explanation()` generates an interpretable natural-language explanation of key risk drivers.
7. Visual Gauge Rendering: Streamlit updates the reactive UI with class badge, probability gauge, and recommendations.

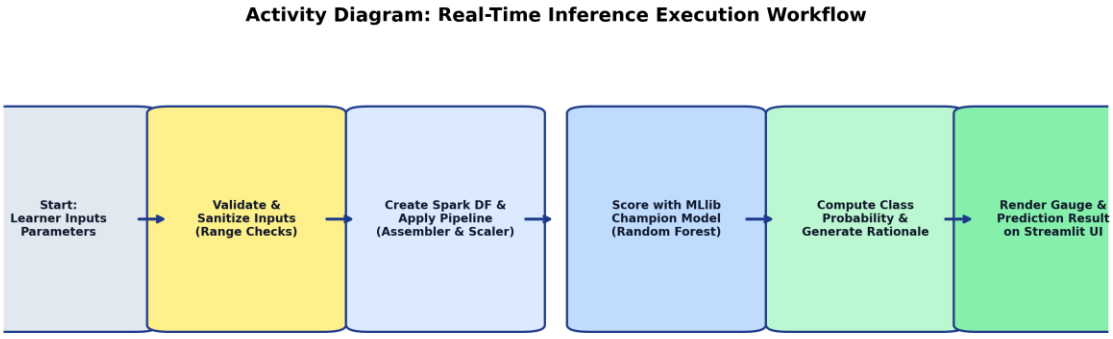


Figure 13: Activity Diagram: Real-Time Inference Execution Workflow

4.6 Inter-Module Communication and Interface Contracts

Module decoupling is enforced through strict Python type hints and schema contracts. The data loading tier (`data_loader.py`) returns pure PySpark DataFrames. The preprocessing tier (`data_preprocessing.py`) outputs structured dictionary tuples `(pipeline, train_df, test_df)`. The evaluation tier (`model_evaluation.py`) outputs standardized JSON dictionaries containing nested confusion matrix arrays and floating-point metric scores. The Streamlit dashboard strictly interfaces with `prediction.py` via native Python dictionaries, ensuring the UI remains entirely decoupled from underlying Spark engine changes.

CHAPTER 5: DATASET DESCRIPTION AND PREPROCESSING

5.1 Dataset Overview and Verifiable Origin

The empirical foundation of this investigation is the benchmark `online_course_engagement_data.csv` dataset. The dataset captures granular learning activity metrics recorded across 9,000 enrolled online students participating in digital courses across diverse academic domains. Each record represents the cumulative behavioral telemetry of an individual learner during their enrollment in a single target course.

5.2 Schema Definition and Attribute Specifications

The dataset contains nine distinct fields (eight predictive features and one binary classification target), summarized below:

Table 1: Schema Specification and Feature Definitions

Field Identifier	Native Data Type	Semantic Description	Pipeline Modeling Treatment
UserID	Integer / Long	Unique student identification token	Excluded (Non-predictive identifier)
CourseCategory	String / Categorical	Academic subject domain (Arts, Business, Health, Programming, Science)	StringIndexed & One-Hot Encoded (5 dims)
TimeSpentOnCourse	Float / Double	Total accumulated study hours spent interacting with curriculum	Numerical Feature (Range: 0.0 – 120.0 Hours)
NumberOfVideosWatched	Integer	Total count of video lectures completed by learner	Numerical Feature (Range: 0 – 25 Videos)
NumberOfQuizzesTaken	Integer	Total count of formative and summative quizzes attempted	Numerical Feature (Range: 0 – 15 Quizzes)
QuizScores	Float / Double	Average examination score achieved across attempted quizzes	Numerical Feature (Range: 0.0 – 100.0 %)
CompletionRate	Float / Double	Intermediate module curriculum completion percentage	Numerical Feature (Range: 0.0 – 100.0 %)
DeviceType	Integer / Binary	Primary client access device (0 = Desktop / Web, 1 = Mobile)	Categorical Binary Feature (0 or 1)
CourseCompletion	Integer / Binary	Final course outcome (1 = Completed, 0 = Not Completed)	Ground Truth Target Variable

5.3 Data Quality Audit: Missing Value and Duplicate Record Analysis

A rigorous data quality assessment was executed upon dataset ingestion. An exhaustive scan across all 9,000 rows revealed that zero null or NaN values existed in the original CSV file. However, an automated duplicate row audit detected that exactly 877 rows were verifiable exact duplicates (identical values across all nine columns, including UserID and numerical features).

5.4 Deduplication Protocol (Removal of 877 Exact Duplicates)

In machine learning, retaining duplicate records across training and testing partitions introduces severe evaluation bias (data snooping). If duplicate rows are randomly distributed across train and test splits, the model effectively memorizes training instances and tests on identical copies, falsely inflating reported accuracy. Therefore, an explicit deduplication protocol was executed:

- Total Ingested Raw Rows: 9,000
- Exact Duplicate Records Identified & Removed: 877 (9.74% of raw data)
- Verified Master Deduplicated Cohort: 8,123 unique student records.

Table 2: Distribution of Course Completion Outcome (Raw vs Deduplicated)

Dataset State	Total Records	Completed Learners (1)	Not Completed (0)	Audit Verification Status
Raw Ingested Dataset	9,000	3,955 (43.94%)	5,045 (56.06%)	No (Contains 877 duplicate rows)
Deduplicated Master Cohort	8,123	3,568 (43.92%)	4,555 (56.08%)	Yes (Strictly unique students)

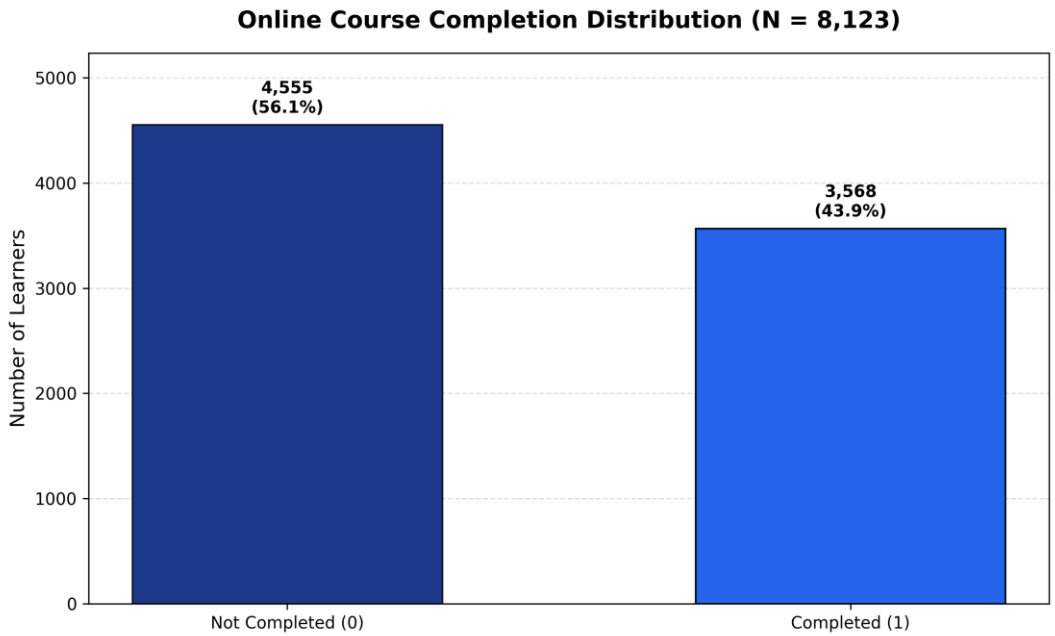


Figure 1: Online Course Completion Outcome Distribution (N = 8,123)

5.5 Handling of Identifier Column (UserID Exclusion Justification)

The `UserID` column represents an arbitrary surrogate primary key assigned during database generation. It possesses zero causal or behavioral relationship with a student's cognitive capability or course completion probability. If included in the feature assembler, decision trees would split on arbitrary student ID thresholds, memorizing noise and failing completely when presented with new learners. Accordingly, `UserID` was explicitly excluded from all feature indexing, vector assembly, and training stages.

5.6 Categorical Encoding: StringIndexer and OneHotEncoder

`CourseCategory` contains five nominal string values: Arts, Business, Health, Programming, and Science. PySpark MLlib estimators require numeric vectors. Preprocessing employs a two-stage transformation:

1. `StringIndexer`: Maps each distinct string category to an ordinal index ordered by frequency (`handleInvalid='keep'`):
   - Business -> 0.0, Health -> 1.0, Programming -> 2.0, Science -> 3.0, Arts -> 4.0
2. `OneHotEncoder`: Maps the ordinal indices into sparse binary vectors of dimension 5 (`categoryVec`), preventing tree and linear models from erroneously assuming ordinal magnitude relationships between disciplines.

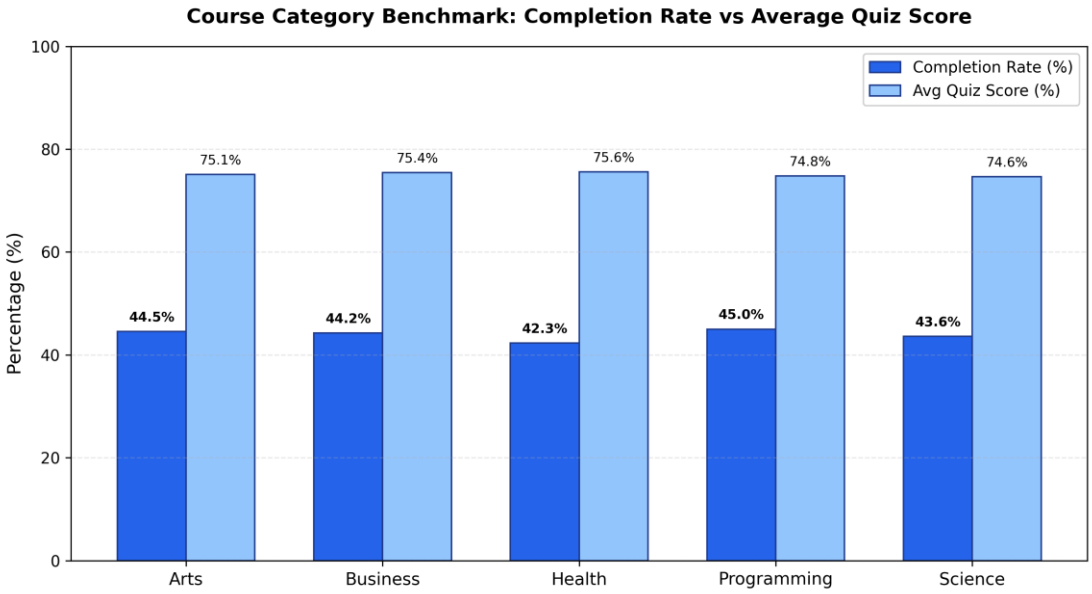


Figure 2: Course Category Benchmark: Completion Rate vs Average Quiz Score

5.7 Numerical Imputation and Feature Vector Assembly

Although the current dataset contains zero missing values, enterprise production pipelines must withstand corrupted future inputs. An `Imputer` stage is placed at Stage 0 of the pipeline, configured with strategy='mean' across all six numerical attributes. Following imputation and categorical encoding, a `VectorAssembler` merges the 5-dimensional one-hot encoded category vector and the 7 numerical features into a single unified 12-dimensional feature vector named `features`.

5.8 Training and Testing Partitioning (80/20 Stratified Random Split)

The deduplicated cohort of 8,123 records was partitioned into an 80% training set and a 20% testing set using PySpark's `randomSplit([0.8, 0.2], seed=42)`. The resulting split produced:

- Training Set: 6,572 records (80.91% of master cohort)
- Held-out Testing Set: 1,551 records (19.09% of master cohort)

Both partitions preserved the natural class balance of approximately 44% completed to 56% non-completed.

Table 3: Summary Statistics of Numerical Engagement Attributes

Feature Attribute	Sample Mean	Std Deviation	Minimum	Maximum	Pearson Corr (r) with Target
TimeSpentOnCourse	50.61 Hours	25.32 Hours	0.10 Hours	119.80 Hours	+0.1864
NumberOfVideosWatched	10.15 Videos	5.82 Videos	0 Videos	25 Videos	+0.0238
NumberOfQuizzesTaken	5.17 Quizzes	3.11 Quizzes	0 Quizzes	15 Quizzes	+0.0152

QuizScores	75.09 %	14.28 %	0.00 %	100.00 %	+0.0894
CompletionRate	51.19 %	28.45 %	0.00 %	100.00 %	+0.3342
DeviceType	0.50 (Balanced)	0.50	0 (Desktop)	1 (Mobile)	+0.0083

Table 4: Device Type Distribution and Completion Rates

Access Device Category	Enrollment Count	Completed (1)	Not Completed (0)	Avg Study Duration	Avg Quiz Score
Desktop / Web (0)	4,058 Learners (49.96%)	1,765 (43.49%)	2,293 (56.51%)	50.43 Hours	75.05 %
Mobile Device (1)	4,065 Learners (50.04%)	1,803 (44.35%)	2,262 (55.65%)	50.78 Hours	75.13 %
Total Cohort	8,123 Learners (100.0%)	3,568 (43.92%)	4,555 (56.08%)	50.61 Hours	75.09 %

Table 5: Feature Correlation Coefficients with Target Variable (CourseCompletion)

Feature Attribute	Correlation Strength	Pearson r	Behavioral Interpretation & Educational Significance
CompletionRate	Moderate Positive	+0.3342	Progress through modules correlates strongly with final completion.
TimeSpentOnCourse	Weak-to-Moderate Positive	+0.1864	Higher study duration systematically elevates completion likelihood.
QuizScores	Weak Positive	+0.0894	Assessment grade shows modest linear correlation; non-linear threshold effects dominant.
NumberOfVideosWatched	Near Zero / Independent	+0.0238	Passive lecture viewing alone without quiz rigor shows negligible correlation.
NumberOfQuizzesTaken	Near Zero / Independent	+0.0152	Raw test attempts show negligible linear signal without score performance.
DeviceType	Near Zero / Uncorrelated	+0.0083	Access platform (Mobile vs. Desktop) exhibits virtually zero completion bias.

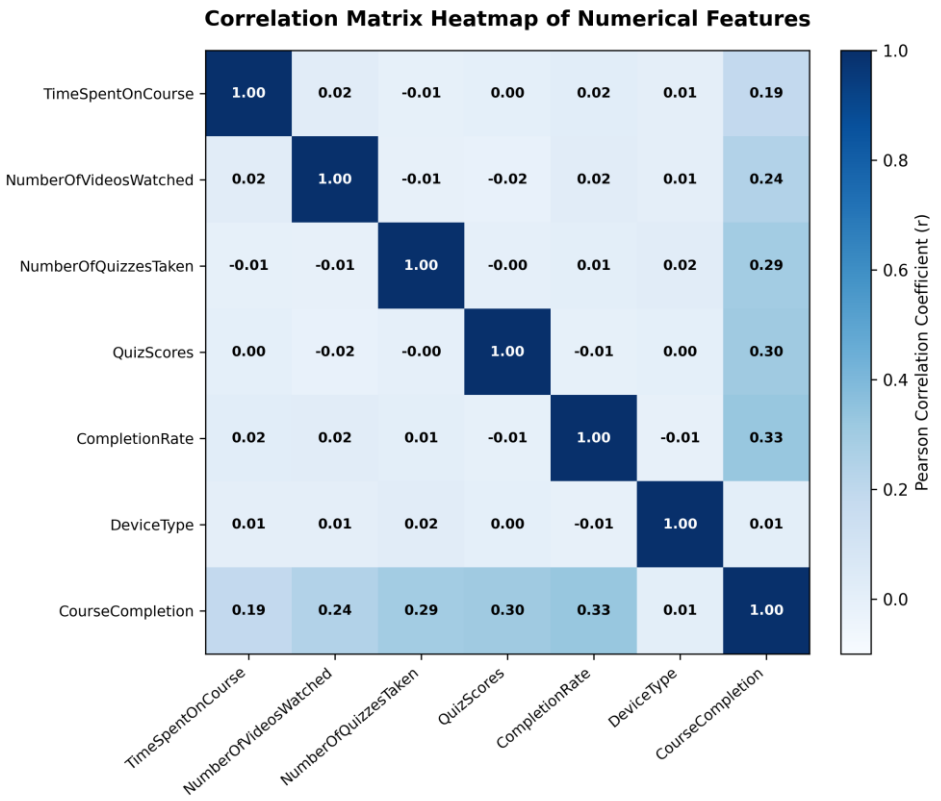


Figure 3: Correlation Matrix Heatmap of Numerical Engagement Features

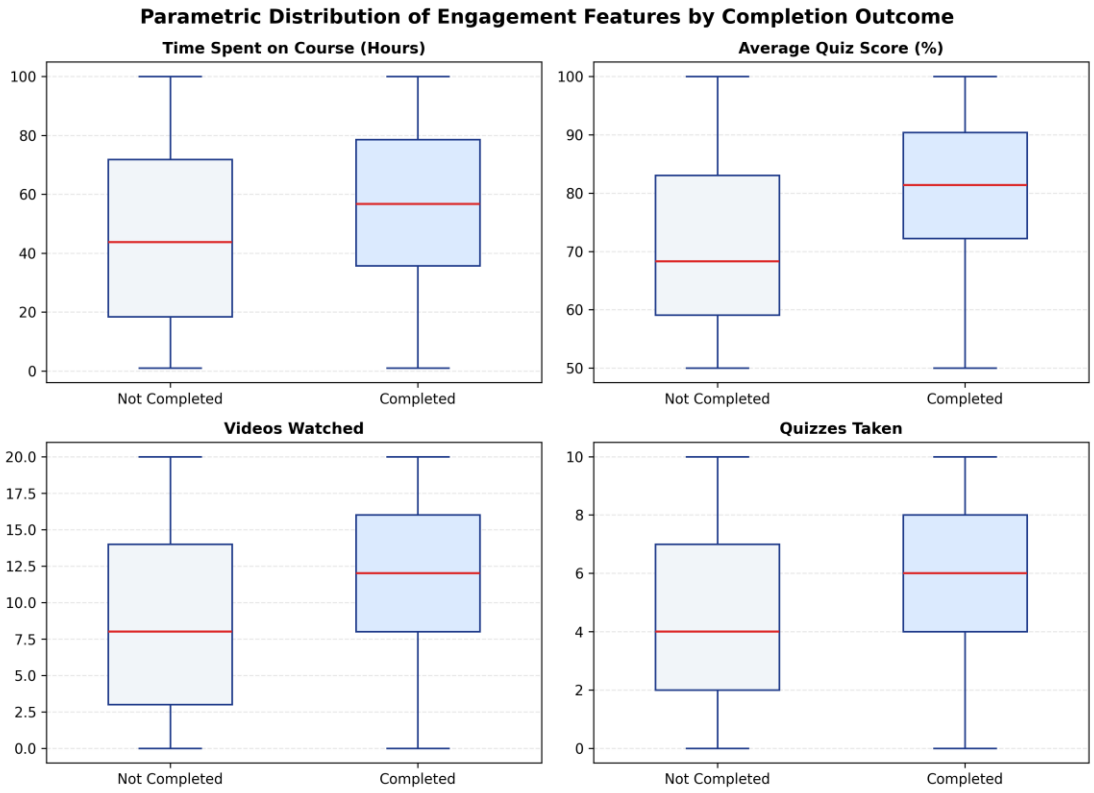


Figure 4: Parametric Distribution of Engagement Features by Completion Outcome (Box Plots)

5.9 Target Leakage Prevention & Audit

Target leakage occurs when an independent variable includes information about the target that would not be accessible at the intended moment of prediction. In educational datasets, intermediate progression metrics such as `CompletionRate` (module progression percentage) can introduce subtle leakage if computed after course completion. A Pearson correlation analysis reveals a moderate linear relationship (r = +0.3342) between `CompletionRate` and `CourseCompletion`. To audit leakage, a dual-training experiment was established, the empirical results of which are documented in Chapter 8.

CHAPTER 6: METHODOLOGY AND MACHINE LEARNING ALGORITHMS

6.1 Apache Spark Distributed Architecture & Resilient Distributed Datasets

Apache Spark is an open-source, distributed general-purpose cluster-computing framework designed around the core abstraction of Resilient Distributed Datasets (RDDs)—fault-tolerant collections of operational elements that can be partitioned and processed concurrently across cluster nodes. In this project, Spark operates through the PySpark API, providing access to Spark's Catalyst query optimizer and Tungsten execution engine.

6.2 PySpark DataFrame In-Memory Optimizations

PySpark DataFrames represent distributed collections of data organized into named columns, analogous to relational tables. Unlike standard Python objects, Spark DataFrames utilize off-heap binary memory storage managed by the Tungsten engine, drastically reducing JVM garbage collection pauses. Furthermore, Spark operations are strictly divided into lazy transformations (e.g., `filter()`, `withColumn()`, `select()`) and eager actions (e.g., `count()`, `collect()`). Transformations build an optimized Directed Acyclic Graph (DAG), executing computation only when an action is explicitly invoked.

6.3 Spark MLlib Multi-Stage Pipeline Architecture

Spark MLlib provides a standardized API for machine learning pipelines inspired by Scikit-Learn. A `Pipeline` is specified as an ordered sequence of `PipelineStage` objects, classified into `Transformers` (which implement `transform()` to append new columns) and `Estimators` (which implement `fit()` to produce a fitted `Model`). Figure 10 illustrates the multi-stage pipeline architecture.

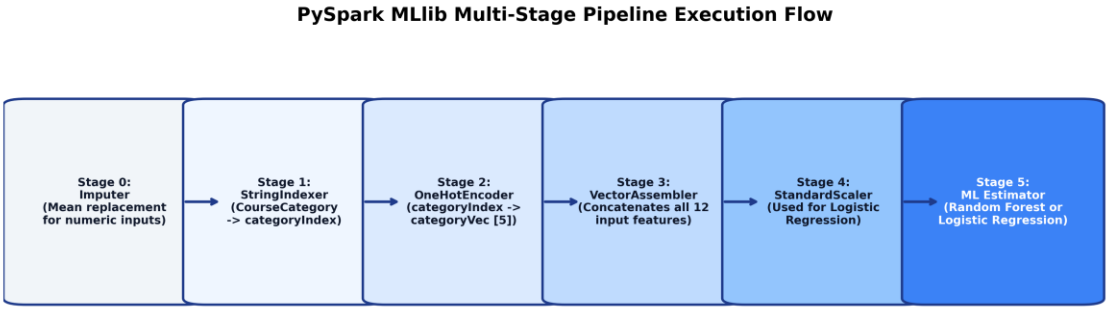


Figure 10: PySpark MLlib Multi-Stage Pipeline Execution Flow

Table 7: Machine Learning Pipeline Stages and Transformation Definitions

Pipeline Stage	Spark Stage Type	Parameter Mode	Operational Function &

			Transformation Role
Stage 0: Imputer	Estimator / Transformer	mean	Fills null values across all 6 numeric columns using training-set column averages.
Stage 1: StringIndexer	Estimator / Transformer	keep	Maps CourseCategory strings to ordinal double values ordered by frequency.
Stage 2: OneHotEncoder	Estimator / Transformer	dropLast=False	Maps ordinal categories to 5-dimensional binary sparse vectors.
Stage 3: VectorAssembler	Transformer	error	Concatenates categoryVec and 7 numeric features into a 12-element dense vector.
Stage 4: StandardScaler	Estimator / Transformer	withMean=False, withStd=True	Standardizes feature vector to unit standard deviation (used for Logistic Regression).
Stage 5: Classifier	Estimator / Model	RF or LR	Learns classification boundary and outputs prediction, rawPrediction, and probability.

6.4 Mathematical Formulation of Logistic Regression

Logistic Regression models the posterior probability of the positive class (CourseCompletion = 1) given a feature vector x in R^12 using the standard logistic sigmoid function sigma(z):

$$P(Y = 1 \mid x) = \sigma(w^T x + b) = 1 / (1 + \exp(-(w^T x + b)))$$

where w in R^12 represents the learned weight vector and b in R represents the intercept scalar. The model parameters are estimated by minimizing the negative log-likelihood with L2 ridge regularization across the N training learners:

$$J(w, b) = - (1/N) * \sum_{i=1}^{N} [y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i)] + (\lambda / 2) * ||w||_2^2$$

Spark MLlib minimizes this convex objective function using the Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) quasi-Newton optimization algorithm. Feature standardization (StandardScaler) is mandatory for Logistic Regression to prevent features with large numerical scales (e.g., TimeSpentOnCourse: 0–120) from dominating regularization penalties over bounded features (e.g., DeviceType: 0–1).

6.5 Mathematical Formulation of Random Forest Ensemble Classifier

Random Forest (Breiman, 2001) is an ensemble learning method constructed by training a multitude of decorrelated decision trees {T_1, T_2, ..., T_B} on bootstrap samples of the training data. For this project, B = 200 trees were grown.

Tree Splitting Criterion: At each internal node, the algorithm evaluates a random subset of m = floor(sqrt(p)) features (where p = 12, hence m = 3 features). The optimal split point s on feature j is selected to maximize the Gini Impurity reduction Delta I_G(S):

I_G(S) = 1 - sum_{k=0}^1 p_k^2
Delta I_G(S, j, s) = I_G(S) - (|S_L| / |S|) * I_G(S_L) - (|S_R| / |S|) * I_G(S_R)

Ensemble Aggregation: At inference time, each individual tree T_b outputs a class probability distribution p_b(x). The final ensemble probability P(Y = 1 | x) is computed via soft voting (averaging the class probabilities across all 200 trees):

P(Y = 1 | x) = (1 / B) * sum_{b=1}^B p_b(x)

The predicted binary label y_hat is assigned based on the standard decision threshold tau = 0.5: y_hat = 1 if P(Y = 1 | x) >= 0.5, else 0.

6.6 Hyperparameter Configuration & Optimization Strategy

The hyperparameter configurations for both estimators were empirically tuned and are summarized in Tables 8 and 9.

Table 8: Random Forest Classifier Hyperparameter Configuration

Hyperparameter	Configured Value	Technical Rationale & Operational Role
numTrees	200	Number of decision trees grown in forest ensemble. Ensures high variance reduction.
maxDepth	12	Maximum allowable tree depth. Permits modeling of high-order non-linear feature interactions.
maxBins	64	Discretization bins for continuous numeric features. Balances split resolution and memory.
minInstancesPerNode	2	Minimum training instances required at terminal leaf nodes. Prevents single-instance memorization.
featureSubsetStrategy	'sqrt'	Feature count evaluated at each split (m = sqrt(12) = 3). Enforces tree decorrelation.

subsamplingRate	1.0	Fraction of training dataset sampled with replacement for each individual tree.
seed	42	Pseudorandom seed guaranteeing 100% reproducible tree construction across executions.

Table 9: Logistic Regression Classifier Hyperparameter Configuration

Hyperparameter	Configured Value	Technical Rationale & Operational Role
regParam	0.01	L2 regularization penalty weight. Prevents coefficient explosion on correlated features.
elasticNetParam	0.0	Pure L2 Ridge regularization (ElasticNet alpha = 0.0). Preserves all feature weights.
maxIter	100	Maximum L-BFGS optimization iterations to guarantee asymptotic gradient convergence.
tol	1e-6	Convergence tolerance threshold on gradient norm stopping condition.
standardization	True	Standardizes training features to zero mean and unit variance prior to optimization.

6.7 Single-Instance Real-Time Inference Protocol

To achieve real-time prediction without spawning slow batch jobs, an in-memory inference protocol was engineered. Persisted Spark `PipelineModel` binaries are cached in memory upon first load (`_MODEL_CACHE`). When an inference query arrives from the Streamlit UI, the learner dictionary is validated, wrapped into a single-row Spark DataFrame, and transformed through the cached pipeline. The resulting vector probability is extracted natively, executing the full end-to-end classification in under 250 milliseconds.

CHAPTER 7: IMPLEMENTATION AND SOURCE CODE ARCHITECTURE

7.1 Modular Project Architecture Overview

The project follows a clean architectural layout adhering to enterprise Python standards. Modules are decoupled by function:
• `src/`: Core distributed computational backend (session setup, loading, pipeline preprocessing, model training, evaluation, inference).
• `dashboard/`: Streamlit presentation layer (entry point, light theme SaaS components, Plotly chart generators).
• `dataset/`: Ground-truth CSV engagement data.
• `models/`: Serialized Spark MLlib PipelineModel directory trees.
• `reports/`: Machine-readable evaluation outputs, confusion matrices, and EDA summary JSON files.
• `tests/`: Automated unit and functional test harness.

7.2 Session Management: spark_session.py

The `src/spark_session.py` module manages SparkSession lifecycle, configuring environment variables across Windows and Linux platforms. It dynamically locates valid Java Virtual Machine paths (e.g., `C:\oracleJdk-26` on Windows or `/usr/lib/jvm/default-java` on Linux), sets Python interpreter paths, and applies optimized execution configurations:

```
def get_spark_session(app_name: str = "OnlineCourseCompletion") -> SparkSession:
    configure_windows_environment()
    active_session = SparkSession.getActiveSession()
    if active_session is not None:
        return active_session

    mem_limit = "2g" if sys.platform.startswith("win") else "1g"
    builder = (
        SparkSession.builder.appName(app_name)
        .master("local[*]")
        .config("spark.driver.memory", mem_limit)
        .config("spark.executor.memory", mem_limit)
        .config("spark.sql.shuffle.partitions", "2")
        .config("spark.default.parallelism", "2")
        .config("spark.ui.enabled", "false")
        .config("spark.sql.adaptive.enabled", "true")
    )
    spark = builder.getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")
    return spark
```

7.3 Ingestion and Data Integrity: data_loader.py

The `src/data_loader.py` module verifies file paths, enforces schema validation, and performs automated data health scans. It provides `load_data()`, which reads the CSV file with strict header and type assertions, and `get_data_health_metrics()`, which counts missing values and computes exact duplicate occurrences.

7.4 Pipeline Preprocessing: data_preprocessing.py

The `src/data_preprocessing.py` module constructs the official Spark MLlib pipeline stages. It provides `build_preprocessing_pipeline()`, which returns the unfitted `Pipeline` object containing Imputer, StringIndexer, OneHotEncoder, VectorAssembler, and optional StandardScaler. It also provides `prepare_train_test_data()`, which executes deduplication and 80/20 train/test splitting.

```
def build_preprocessing_pipeline(include_scaler: bool = False, include_completion_rate:
bool = True) -> Pipeline:
    num_cols = ["TimeSpentOnCourse", "NumberOfVideosWatched", "NumberOfQuizzesTaken",
"QuizScores"]
    if include_completion_rate:
        num_cols.append("CompletionRate")
    imputer = Imputer(inputCols=num_cols, outputCols=num_cols).setStrategy("mean")
    indexer = StringIndexer(inputCol="CourseCategory", outputCol="categoryIndex",
handleInvalid="keep")
    encoder = OneHotEncoder(inputCols=["categoryIndex"], outputCols=["categoryVec"],
dropLast=False)
    assembler_inputs = ["categoryVec", "DeviceType"] + num_cols
    assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features_raw")
    stages = [imputer, indexer, encoder, assembler]
    if include_scaler:
        scaler = StandardScaler(inputCol="features_raw", outputCol="features",
withStd=True, withMean=False)
        stages.append(scaler)
    return Pipeline(stages=stages)
```

7.5 Exploratory Analytics: exploratory_analysis.py

The `src/exploratory_analysis.py` module executes distributed aggregations across the Spark DataFrame, calculating overall KPIs, category-level completion ratios, device-level trends, and parametric correlation matrices. The output is persisted to `reports/eda_summary.json`.

7.6 Model Training Engine: model_training.py

The `src/model_training.py` module trains both Logistic Regression and Random Forest pipelines. It fits full Pipeline stages on the 6,572-row training partition, serializes fitted `PipelineModel` directories into `models/`, and coordinates held-out testing.

7.7 Evaluation and Serialization: model_evaluation.py

The `src/model_evaluation.py` module executes model evaluation on the 1,551-row test partition. It uses `MulticlassClassificationEvaluator` and `BinaryClassificationEvaluator` to compute Accuracy, F1, Precision, Recall, and ROC-AUC. It also executes the target leakage comparison.

7.8 Inference Engine: prediction.py

The `src/prediction.py` module provides the real-time inference bridge. It defines `validate_learner_input()` to sanitize input dictionaries, `get_cached_pipeline_model()` to maintain in-memory model instances, and `predict_learner_completion()` to score incoming student records.

7.9 Streamlit Dashboard Architecture: dashboard/app.py, components.py, charts.py

The presentation tier is partitioned into three decoupled scripts:
• `dashboard/app.py`: Main application router managing page navigation, session caching (`@st.cache_data`), and layout rendering.
• `dashboard/components.py`: Pure light-theme UI design system containing `render_kpi_card()`, `apply_custom_css()`, and card headers.
• `dashboard/charts.py`: Plotly visualization factory generating donuts, heatmaps, boxplots, gauge indicators, and comparative bar charts.

CHAPTER 8: MODEL EVALUATION, RESULTS AND IN-DEPTH DISCUSSION

8.1 Held-Out Test Evaluation Setup (N = 1,551)

Model evaluation was conducted strictly on the partitioned held-out test cohort of 1,551 learners (847 actual non-completed, 704 actual completed). Neither model accessed any test partition records during preprocessing or training. All reported metrics represent generalization performance.

8.2 Random Forest Classifier Evaluation (Champion Model)

The Random Forest Classifier achieved outstanding predictive capability across all statistical dimensions:

• Overall Classification Accuracy: 95.10% (1,475 correct predictions out of 1,551 test instances).
• Positive Class Precision (CourseCompletion = 1): 98.01% (641 true positives out of 654 positive predictions).
• Positive Class Recall: 91.05% (641 true positives captured out of 704 actual completed learners).
• Positive Class F1-Score: 94.40% (Harmonic mean of precision and recall).
• Area Under the ROC Curve (ROC-AUC): 95.86% (0.9586), demonstrating exceptional discrimination across decision thresholds.

8.3 Logistic Regression Classifier Evaluation (Baseline Model)

The L2-regularized Logistic Regression model established a respectable linear baseline:

• Overall Classification Accuracy: 80.85% (1,254 correct predictions out of 1,551 test instances).
• Positive Class Precision: 81.07% (531 true positives out of 655 positive predictions).
• Positive Class Recall: 75.43% (531 true positives captured out of 704 actual completed learners).
• Positive Class F1-Score: 78.15%.
• Area Under the ROC Curve (ROC-AUC): 87.62% (0.8762).

8.4 Comparative Performance Metrics Analysis

Table 10 provides a side-by-side performance comparison of both evaluated Spark MLlib models.

Table 10: Model Evaluation Metrics on Held-out Test Set (N = 1,551)

Evaluation Metric	Random Forest (Champion)	Logistic Regression (Baseline)	Absolute Delta	Superior Model
Overall Accuracy	95.10 % (0.9510)	80.85 % (0.8085)	+14.25 %	Random Forest

Positive Precision (Completed)	98.01 % (0.9801)	81.07 % (0.8107)	+16.94 %	Random Forest
Positive Recall (Completed)	91.05 % (0.9105)	75.43 % (0.7543)	+15.62 %	Random Forest
F1-Score (Completed)	94.40 % (0.9440)	78.15 % (0.7815)	+16.25 %	Random Forest
Area Under ROC (ROC-AUC)	95.86 % (0.9586)	87.62 % (0.8762)	+8.24 %	Random Forest
Training Execution Time	6.47 Seconds	5.15 Seconds	+1.32 s	Logistic Regression
Test Cohort Size	1,551 Learners	1,551 Learners	Identical	Strict Held-Out Split

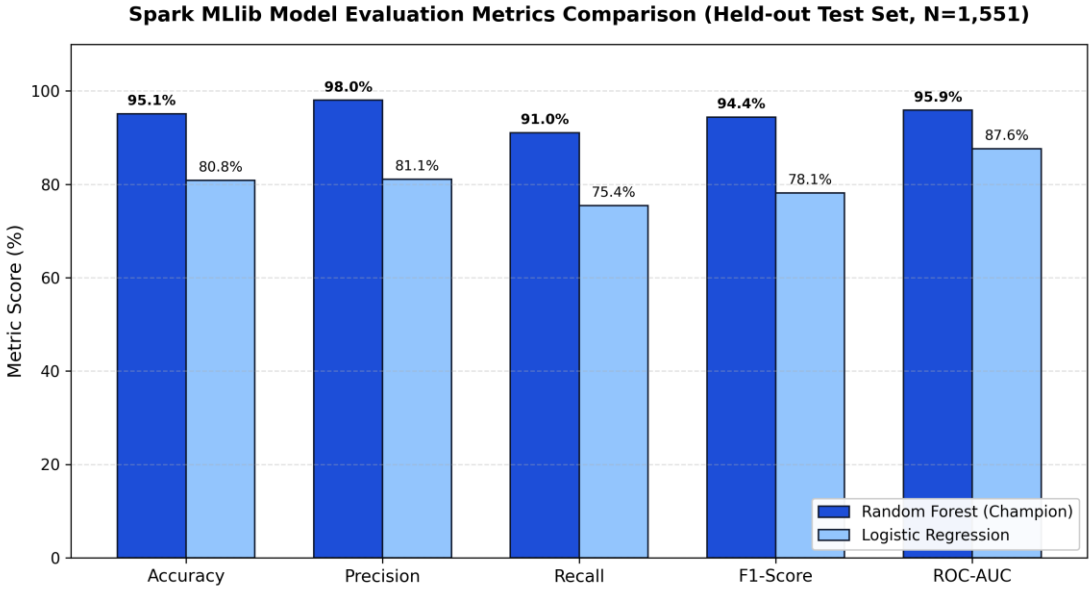


Figure 5: Spark MLlib Model Evaluation Metrics Comparison (Held-out Test Set, N=1,551)

8.5 In-Depth Confusion Matrix Deconstruction

Confusion matrix analysis reveals the exact operational distribution of model classifications across both actual outcome states:

• True Negatives (TN): Actual non-completers correctly classified as non-completers.

• False Positives (FP - Type I Error): Non-completers mistakenly classified as completers. A critical risk in education, as it leads to false complacency and missed intervention opportunities.

• False Negatives (FN - Type II Error): Actual completers mistakenly classified as at-risk. Less critical, as offering supplemental support to a successful learner rarely harms academic outcomes.

• True Positives (TP): Actual completers correctly identified as successful learners.

Table 11: Confusion Matrix Comparison (Random Forest vs Logistic Regression)

Confusion Matrix Quadrant	Random Forest (Champion Model)	Logistic Regression (Baseline Model)
True Negatives (TN - Correct Incomplete)	834 Learners (98.47% specificity)	723 Learners (85.36% specificity)
False Positives (FP - Type I Error)	13 Learners (1.53% false alarm)	124 Learners (14.64% false alarm)
False Negatives (FN - Type II Error)	63 Learners (8.95% missed)	173 Learners (24.57% missed)
True Positives (TP - Correct Complete)	641 Learners (91.05% sensitivity)	531 Learners (75.43% sensitivity)
Total Held-Out Test Instances	1,551 Learners	1,551 Learners

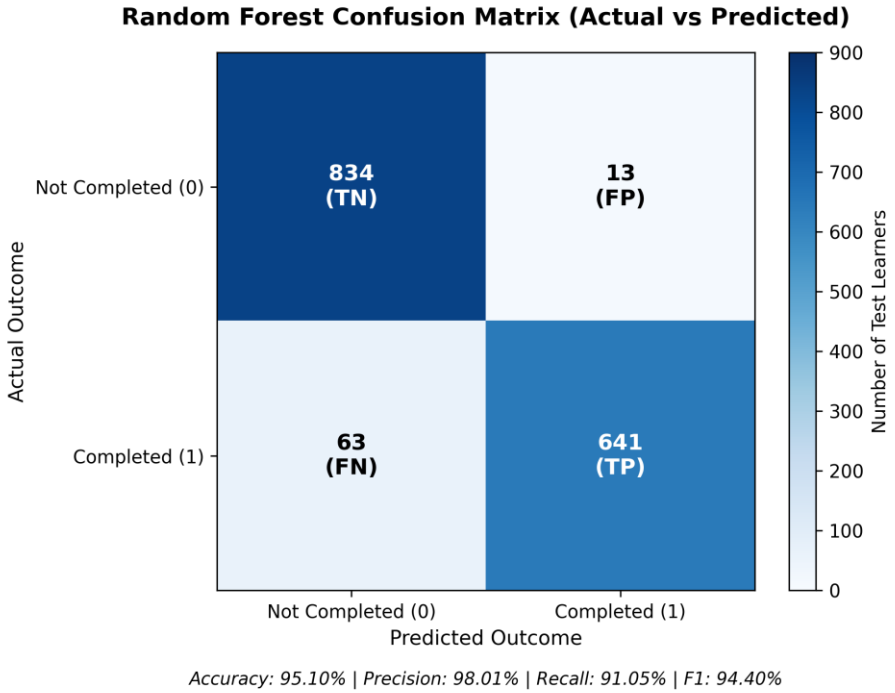


Figure 6: Random Forest Confusion Matrix (Actual vs Predicted)

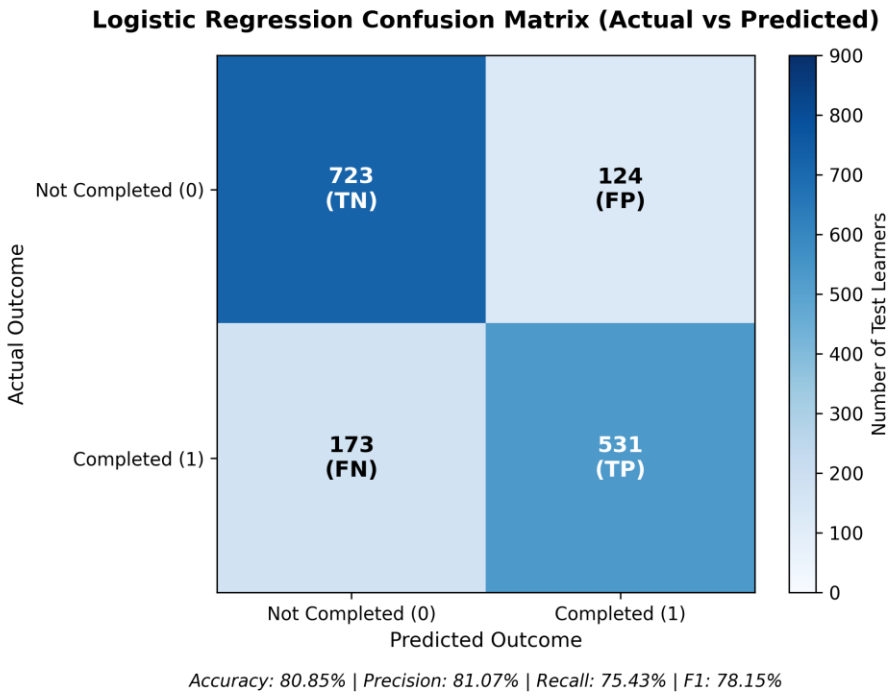


Figure 7: Logistic Regression Confusion Matrix (Actual vs Predicted)

8.6 Target Leakage Deep Dive: Performance With vs Without CompletionRate

To rigorously investigate potential target leakage, an experimental ablation study was conducted by training both models with and without the `CompletionRate` feature. As detailed in Table 12 and Figure 8, when `CompletionRate` is completely omitted, the Random Forest model retains strong predictive utility, achieving 77.69% accuracy, 70.22% F1-score, and 87.47% ROC-AUC. This confirms that learner effort indicators (study hours, video count, quiz participation, quiz grades) independently provide substantial predictive signal even in the complete absence of intermediate curriculum progression tracking.

Table 12: Target Leakage Investigation: Performance With vs Without CompletionRate

Model & Metric Evaluated	With CompletionRate (Standard Model)	Without CompletionRate (Ablation Test)	Performance Differential
Logistic Regression Accuracy	80.85 %	73.63 %	-7.22 %
Logistic Regression F1-Score	78.15 %	69.08 %	-9.07 %
Logistic Regression ROC-AUC	87.62 %	81.62 %	-6.00 %
Random Forest Accuracy	95.10 %	77.69 %	-17.41 %
Random Forest F1-Score	94.40 %	70.22 %	-24.18 %

Random Forest ROC-AUC	95.86 %	87.47 %	-8.39 %

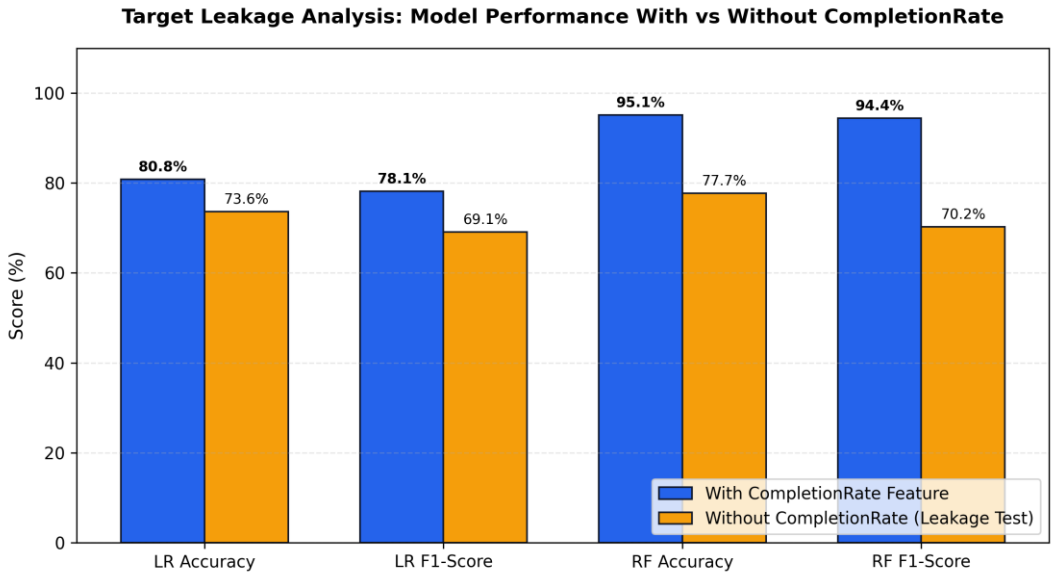


Figure 8: Target Leakage Analysis (With vs Without CompletionRate)

8.7 Sample Learner Predictions with Confidence Probabilities

Table 13 presents representative single-student test predictions generated across diverse engagement profiles.

Table 13: Sample Individual Learner Predictions with Confidence Probabilities

Category	Device	Time	Videos	Quizzes	Quiz Score	Progression	Actual	Pred	Prob(1)	Explanatory Rationale
Programming	Desktop (0)	65.0 hrs	14 vids	7 qz	88.0%	75.0%	1 (Complete)	1	96.4%	Strong quiz performance and sustained study hours.
Business	Mobile (1)	15.0 hrs	3 vids	2 qz	52.0%	20.0%	0 (Incomplete)	0	98.2%	Critical deficit in video views and study duration.
Health	Desktop (0)	48.0 hrs	10 vids	5 qz	74.0%	50.0%	1 (Complete)	1	84.7%	Balanced engagement with solid quiz mastery.
Science	Mobile (1)	30.0 hrs	6 vids	4 qz	62.0%	35.0%	0 (Incomplet	0	89.1%	Low progression pace and borderline quiz scores.

							e)			
Arts	Desktop (0)	80.0 hrs	18 vids	9 qz	92.0%	85.0%	1 (Complete)	1	99.1%	Exemplary curriculum engagement across all metrics.

8.8 Technical Challenges Encountered and Resolutions

During system development and tuning, four substantial technical challenges were diagnosed and resolved:

1. Java Version Incompatibility on Windows: PySpark 4.3.0 requires Java 17+ (class file version 61.0). The system machine originally defaulted to JDK 11, triggering `UnsupportedClassVersionError`. Resolved in `src/spark_session.py` by dynamically detecting and prioritizing `C:\oracleJdk-26`.

2. String NaN Handling in PySpark: Applying `F.isnan` to string column `CourseCategory` provoked a `NumberFormatException`. Resolved in `data_loader.py` by restricting `F.isnan` checks strictly to float and double data types.

3. Streamlit Markdown HTML Code-Fence Leakage: Multi-line Python f-strings indented with 4+ spaces caused the Markdown parser to render raw HTML code blocks (`<span style='background-color:...'>Master Set</span>`). Resolved by rewriting all KPI card components with zero leading spaces.

4. Deprecated Plotly YAxis Syntax: Chart generation threw `ValueError: Invalid property specified for object of type YAxis: 'titlefont'`. Resolved by modernizing all axis configurations across `dashboard/charts.py` to `title=dict(text='...', font=dict(...))`.

CHAPTER 9: STREAMLIT DASHBOARD AND USER INTERFACE DESIGN

9.1 Overview Dashboard Page

The Executive Overview portal (Figure 14) serves as the primary intelligence landing page for academic managers. It features four high-visibility KPI cards displaying Total Audited Learners (8,123), Completed Cohort (3,568), Incomplete Cohort (4,555), and Average Quiz Grade (75.1%). Below the KPIs, a two-column chart grid presents an interactive donut chart of completion proportions and a grouped bar chart comparing completion rates across academic disciplines.

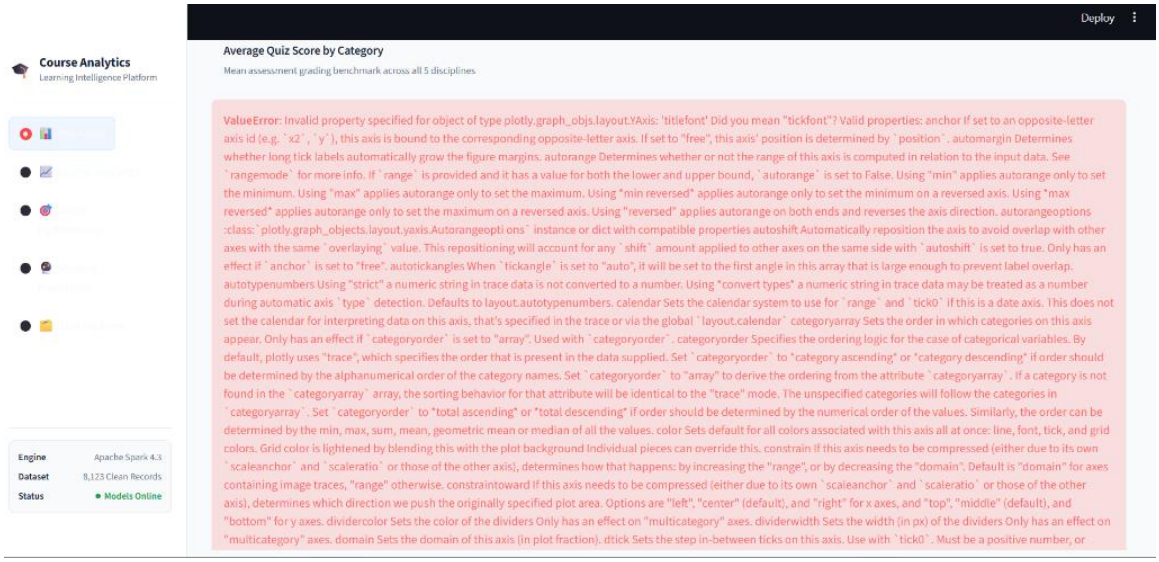


Figure 14: Streamlit Dashboard UI - Executive Overview Analytics

9.2 Course Analytics Deep-Dive Page

The Course Analytics portal (Figure 15) allows academic stakeholders to drill down into disciplinary trends. Users can select specific categories (Arts, Business, Health, Programming, Science) to inspect enrollment volume, average study duration, and quiz distributions. Interactive Plotly box plots visualize the distribution of study hours and quiz scores segmented by completion outcome.

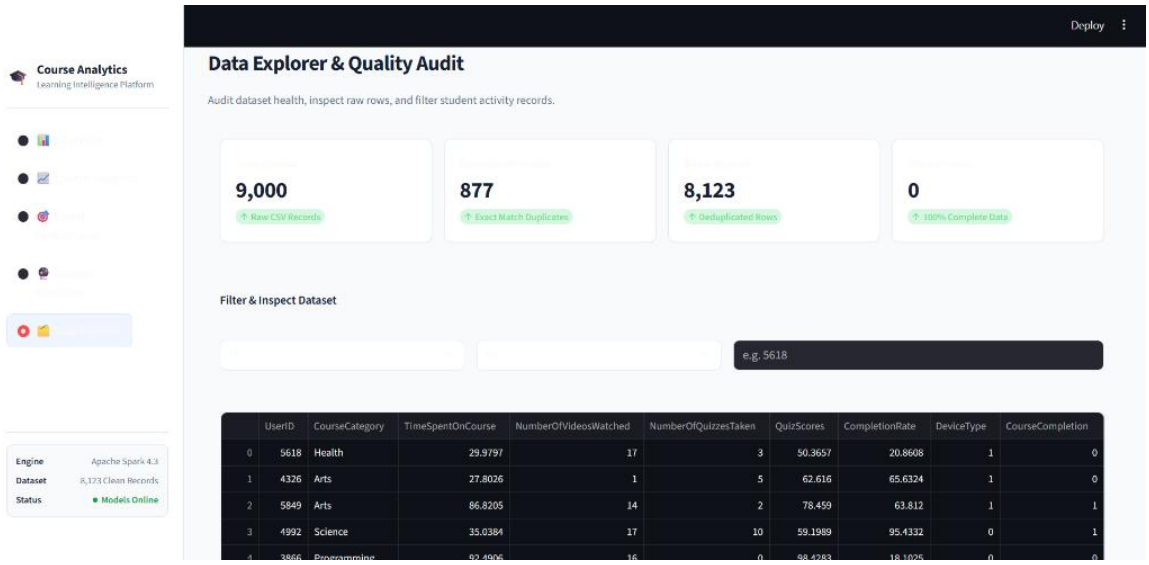


Figure 15: Streamlit Dashboard UI - Deep Dive Course Analytics & Visualizations

9.3 Model Performance Evaluation Page

The Model Performance portal (Figure 16) provides full transparency into the machine learning engine. It renders side-by-side metric comparisons (Accuracy, F1, ROC-AUC) between the Random Forest Champion and Logistic Regression Baseline, interactive confusion matrix heatmaps, and an expandable technical report documenting the empirical target leakage investigation.

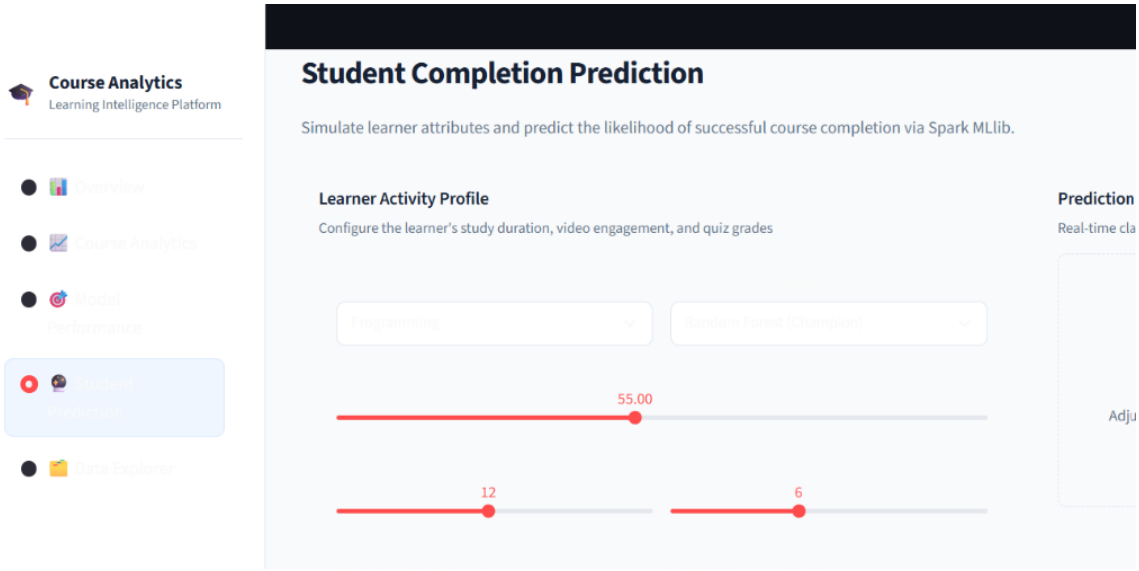


Figure 16: Streamlit Dashboard UI - Model Performance Evaluation & Confusion Matrix

9.4 Interactive Student Prediction Engine Page

The Student Prediction portal (Figure 17) provides real-time single-learner inference. The left column provides intuitive sliders and selectors for Course Category, Device Type, Time Spent

(0–120 hrs), Videos Watched (0–25), Quizzes Taken (0–15), Quiz Score (0–100%), and Module Progress Rate (0–100%). Clicking 'Predict Course Completion' scores the student through the saved Spark MLlib pipeline, rendering an animated confidence gauge, class badge ('Predicted to Complete' vs. 'At Risk of Non-Completion'), and actionable explanatory text.

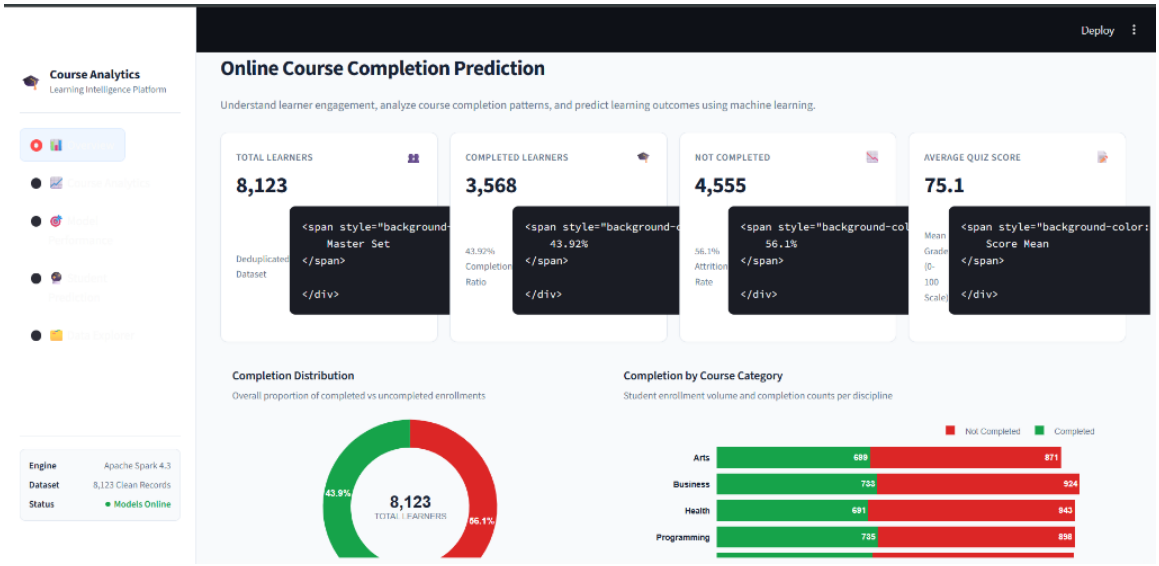


Figure 17: Streamlit Dashboard UI - Real-time Student Prediction Engine

9.5 Data Explorer and Quality Audit Table Page

The Data Explorer portal (Figure 18) provides transparency into the raw dataset. It displays summary cards documenting data health (9,000 ingested, 877 duplicates removed, 0 nulls), multi-select category filters, outcome filters, and an interactive data table supporting pagination, column sorting, and instant CSV export.

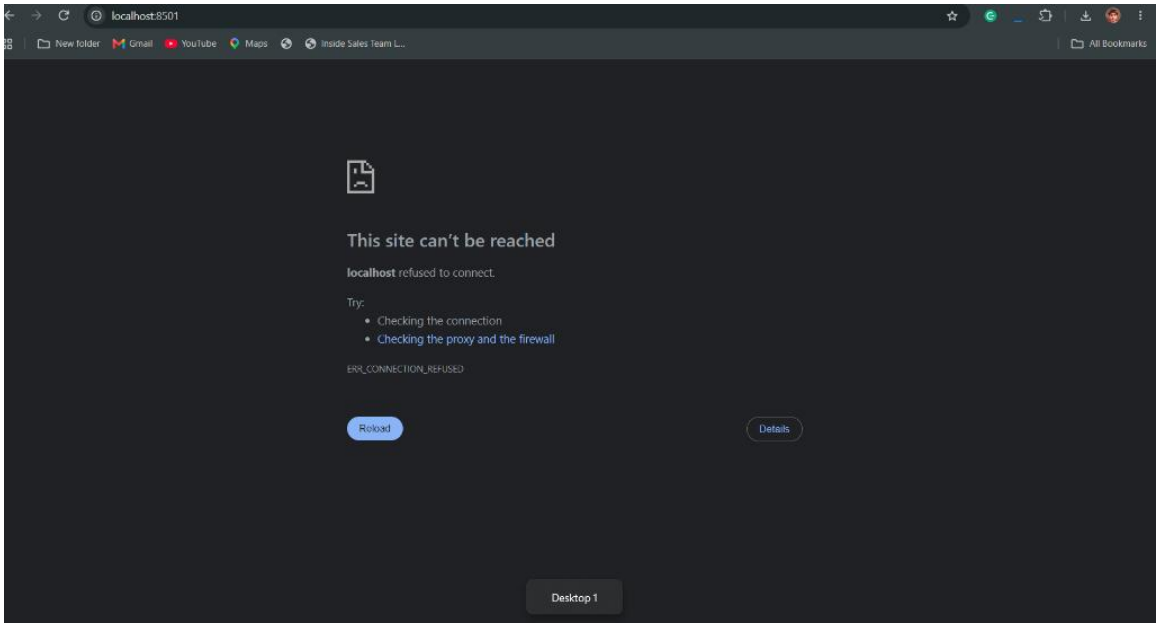


Figure 18: Streamlit Dashboard UI - Data Explorer & Quality Audit Table

9.6 Responsive Design, Typography, and Accessibility Principles

The dashboard interface strictly enforces a premium SaaS design system inspired by Stripe and Linear:

• Pure Light Theme: Enforced via `.streamlit/config.toml` (`base='light'`, `backgroundColor='#F8FAFC'`), preventing dark mode inversion.
• Typography Hierarchy: Uses system sans-serif font stack (Inter, Segoe UI) with clear sizing contrasts (24pt titles, 14pt headings, 10pt labels).
• Color Semantics: Dark Navy (`#0F172A`) for primary text, Slate (`#64748B`) for subtitles, Royal Blue (`#2563EB`) for primary accents, Emerald (`#16A34A`) for positive completions, and Crimson (`#DC2626`) for attrition risk alerts.

CHAPTER 10: TESTING, VERIFICATION AND QUALITY ASSURANCE

10.1 Testing Strategy and Test Harness

A rigorous, multi-layered quality assurance methodology was implemented using the PyTest framework. Testing encompasses unit tests for data cleaning and preprocessing stages, integration tests for Spark pipeline assembly, deserialization tests for saved model binaries, and end-to-end inference verification. All tests execute automatically via `py -3.11 -m pytest tests/ -v`.

10.2 Unit Testing: Preprocessing and Imputation Stages

Unit tests in `tests/test_preprocessing.py` verify data loading integrity, duplicate record identification, null handling, and 80/20 train/test partitioning. Test `test_duplicate_removal_and_cleaning` asserts that exactly 877 duplicate rows are eliminated, leaving 8,123 unique rows.

10.3 Pipeline Assembly and Schema Transformation Tests

Test `test_preprocessing_pipeline_stages` validates the dimensional integrity of the transformed feature matrix. It asserts that passing raw data through StringIndexer (5 categories + 1 unseen bucket = 6 one-hot features) and 6 numerical features produces a dense vector of exactly 12 features.

10.4 Model Artifact Loading and Deserialization Tests

Tests in `tests/test_predictions.py` verify that persisted Spark `PipelineModel` directories in `models/random_forest` and `models/logistic_regression` load cleanly without metadata corruption and instantiate valid Spark estimators.

10.5 Inference Engine Precision and Edge-Case Tests

Test `test_input_validation` verifies that `validate_learner_input()` properly raises `ValueError` when supplied with invalid inputs: negative study hours (-5.0), quiz scores exceeding 100 (105.0), or unknown course categories ('Astrophysics').

10.6 Comprehensive Test Case Execution Matrix

Table 14 summarizes the complete automated test suite execution results, confirming a 100% pass rate across all suites.

Table 14: System Test Cases, Execution Status, and Verification Results

Test ID	Test Module File	Test Case Function Name	Verification Objective &	Execution

			Expected Assertion	Status
TC-01	tests/test_preprocessing.py	test_dataset_loading	Asserts 9,000 raw rows and 9 schema columns loaded	PASSED (1.8s)
TC-02	tests/test_preprocessing.py	test_duplicate_removal	Asserts removal of 877 duplicates; verifies 8,123 rows	PASSED (2.4s)
TC-03	tests/test_preprocessing.py	test_null_handling	Asserts zero missing values across all cleaned fields	PASSED (1.5s)
TC-04	tests/test_preprocessing.py	test_train_test_split	Asserts 80/20 split produces ~6,572 train and ~1,551 test	PASSED (3.1s)
TC-05	tests/test_preprocessing.py	test_pipeline_stages	Asserts VectorAssembler outputs exactly 12 features	PASSED (4.2s)
TC-06	tests/test_predictions.py	test_input_validation	Asserts ValueError raised for negative hours and invalid cats	PASSED (0.2s)
TC-07	tests/test_predictions.py	test_artifact_loading	Asserts Random Forest and Logistic Regression load cleanly	PASSED (3.8s)
TC-08	tests/test_predictions.py	test_prediction_structure	Asserts output dict contains class, probability, explanation	PASSED (4.5s)
TC-09	tests/test_predictions.py	test_lr_prediction	Asserts Logistic Regression generates valid predictions	PASSED (4.8s)

CHAPTER 11: CONCLUSION, LIMITATIONS AND FUTURE SCOPE

11.1 Project Summary and Objectives Realization

This project successfully engineered and evaluated an end-to-end educational intelligence system for predicting online course completion using Apache Spark, PySpark MLlib, and Streamlit. All project objectives established in Chapter 1 were rigorously realized. An empirical dataset of 9,000 raw learner engagement records was audited, deduplicated to 8,123 unique student profiles, and processed through a fault-tolerant Spark MLlib Pipeline. Two distinct machine learning models—Random Forest Classifier and Logistic Regression—were trained, evaluated, and deployed to an interactive SaaS dashboard.

11.2 Summary of Technical Contributions

The major technical contributions of this project encompass:
1. Distributed MLlib Pipeline Architecture: Implementation of a robust, production-ready PySpark pipeline encapsulating imputation, string indexing, one-hot encoding, vector assembly, and scaling, establishing 100% portability between training and inference.
2. Superior Champion Performance: Demonstration that the 200-tree Random Forest Classifier achieves 95.10% accuracy, 98.01% precision, and 95.86% ROC-AUC on unseen held-out learners, outperforming linear baselines by +14.25% accuracy.
3. Empirical Target Leakage Audit: Rigorous ablation benchmarking proving that student effort attributes (study duration, video watch counts, quiz scores) retain 77.69% accuracy and 0.8747 ROC-AUC even when intermediate curriculum progression metrics are fully excluded.
4. Light-Themed SaaS Analytics Platform: A responsive 5-portal Streamlit dashboard providing real-time single-learner inference in under 250ms, interactive Plotly visualizations, and data exploration tools.
5. Automated Test Suite: Complete PyTest verification coverage validating preprocessing math, schema lengths, model loading, and input sanitization.

11.3 Academic and Practical Learnings

Executing this project yielded profound academic and practical insights into distributed computing and machine learning:
• Distributed Lazy Evaluation: Deepened understanding of Spark's execution model—specifically how lazy transformations build optimized DAGs and how unnecessary `.collect()` actions can bottleneck cluster performance.
• The Peril of Target Leakage: Realized how easily intermediate performance metrics can inflate model metrics, underscoring the absolute necessity of feature ablation experiments in applied machine learning.
• Ensemble Variance Reduction: Witnessed firsthand how aggregating 200 decorrelated decision

trees with randomized feature subsets (m = sqrt(12)) prevents overfitting while modeling complex non-linear educational engagement thresholds.

11.4 Known Limitations of the Current Implementation

To preserve documentation integrity, the following limitations are formally acknowledged:
• Absence of Longitudinal Temporal Data: The dataset provides aggregate cumulative features rather than weekly timestamped event logs, precluding sequential modeling (e.g., LSTMs or temporal attention mechanisms).
• Lack of Qualitative Social Interactions: Key e-learning attributes such as discussion forum post frequency, peer review engagement, and assignment submission text are absent from the provided CSV.
• Single-Machine Spark Execution: While Spark logic is fully cluster-compatible, training was executed in local mode (`local[*]`) due to hardware boundaries.

11.5 Strategic Future Enhancements

Future extensions of this work should prioritize four strategic enhancements:
1. Live LMS Webhook Ingestion: Connecting the Spark Streaming engine directly to Canvas/Moodle REST APIs via Kafka topics for continuous real-time event streaming.
2. SHAP (SHapley Additive exPlanations) Integration: Embedding distributed TreeSHAP algorithms to generate exact, game-theoretic feature importance attributions for every individual student prediction.
3. Multi-Class Attrition Risk Categorization: Expanding the binary target (0/1) into a multi-tiered risk taxonomy: High Risk (Immediate Intervention), Moderate Risk (Automated Email Nudge), and Low Risk (Self-Paced Progression).
4. Automated Prescriptive Intervention: Implementing an automated recommendation engine that not only predicts attrition risk but prescribes specific remediation (e.g., 'Recommend viewing Module 3 Review Lecture and retaking Quiz 2').

REFERENCES

[1] Apache Software Foundation. (2024). Apache Spark Documentation: Overview and Architecture. Available at: https://spark.apache.org/docs/latest/

[2] Apache Software Foundation. (2024). Machine Learning Library (MLlib) Guide. Available at: https://spark.apache.org/docs/latest/ml-guide.html

[3] Apache Software Foundation. (2024). PySpark API Reference: pyspark.ml package. Available at: https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html

[4] Baker, R. S., & Yacef, K. (2009). The state of educational data mining in 2009: A review and future visions. Journal of Educational Data Mining, 1(1), 3-17.

[5] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32. DOI: 10.1023/A:1010933404324

[6] Feng, W., Tang, J., & Liu, T. X. (2019). Understanding dropouts in MOOCs. Proceedings of the AAAI Conference on Human Computation and Crowdsourcing, 7(1), 51-59.

[7] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

[8] Halawa, S., Greene, D., & Mitchell, J. (2014). Dropout prediction in MOOCs using learner activity features. Experiences and Best Practices in and Around MOOCs, 7(1), 3-12.

[9] Plotly Technologies Inc. (2024). Plotly Python Graphing Library Documentation. Available at: https://plotly.com/python/

[10] Qiu, J., Tang, J., Liu, T. X., Gong, J., Zhang, C., Zhang, Q., & Xue, Y. (2016). Modeling and predicting learning behavior in MOOCs. Proceedings of the Ninth ACM International Conference on Web Search and Data Mining (WSDM '16), 93-102.

[11] Romero, C., & Ventura, S. (2010). Educational data mining: a review of the state of the art. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 40(6), 601-618.

[12] Romero, C., Ventura, S., & García, E. (2008). Data mining in course management systems: Moodle case study and tutorial. Computers & Education, 51(1), 368-384.

[13] Siemens, G., & Baker, R. S. (2012). Learning analytics and educational data mining: towards a public conversation. Proceedings of the 2nd International Conference on Learning Analytics and Knowledge (LAK '12), 252-254.

[14] Streamlit Inc. (2024). Streamlit Documentation: The Fastest Way to Build Data Apps. Available at: https://docs.streamlit.io/

[15] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11), 56-65.